Computational Control

Lecture Handout

April 15, 2026

Contents

1 Dynamic Programming and LQR

1.1 Introduction

In general we formulate the optimization control task as

$$\min_{\mathbf{u}\in\mathcal{U}} J(\mathbf{u}) \tag{1}$$

where $\mathcal{U} = \tilde{\mathcal{U}} \times \{0, 1, \dots, \mathrm{T}\}$ is the set of the feasible inputs and $J(\mathbf{u})$ is generally the sum of some *stage costs* and a terminal cost/constraint.
The issue is that generally 1 is an intractable optimization problem. This is due to several factors:

- the dimension of $\mathbf{u}$ is too big;
- the cost function requires an accurate model of the system;
- the optimization problem is **non-convex**;
- In presence of **disturbances** and **uncertainity** the open-loop nature of the task makes it useless

In this context, Dynamic Programming (DP) is a powerfull tool whenever we can decompose the problem. In particular when the problem allows for a **Markovian representation**:

Key assumptions: Markovian representation

The following hypothesis are crucial in order for DP to be meaningfull:

1. There exists a **Markovian State**[a] that evolves according to some dynamics
$$x_{t+1} = f_t(x_t, u_t)^{[b]}$$
2. The initial condition x_0 is known.
3. The cost is additive over time, i.e.
$$J = \sum_t g_t(x_t, u_t)$$

[a] i.e. $X_{t+1} \perp X_{1:t-1} | X_t$
[b] discrete time dynamical system representation. Notice f_t can change with time

In this context a powerfull result is given by **Bellman's principle**. Considering the following setup:

$$\begin{aligned} \min_{\mathbf{u},\mathbf{x}} \quad & \sum_{t=0}^{T} g_t(x_t, u_t) \\ \text{subject to} \quad & x_{t+1} = f_t(x_t, u_t) \\ & x_0 = X_0 \\ & x_t \in \mathcal{X}_t \ \forall t \\ & u_t \in \mathcal{U}_t \ \forall t. \end{aligned}$$

a very powerfull result on which relies DP is **Bellman's optimality principle**.

Bellman's optimality principle

An optimal policy has the property that whatever the initial state and the initial decisions are, the remaining decisions must consititute an optimal policy with regard to the state resulting from the first decision.

In a intuitive way we can say that any tail of an optimal trajectory is optimal too.

$$\begin{aligned} V_0(X_0) &= \min_{u_0} \left\{ g_0(X_0, u_0) + \min_{\substack{x_1,\dots,x_T \\ u_1,\dots,u_T}} \sum_{t=1}^{T} g_t(x_t, u_t) \;\text{ s.t. } \begin{cases} x_{t+1} = f_t(x_t, u_t) \\ x_1 = f_0(X_0, u_0) \end{cases} \right\} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(x_1) \} \\ &= \min_{u_0} \{ g_0(X_0, u_0) + V_1(f_0(x_0, u_0)) \} \end{aligned}$$

We can see that the problems are nested, therefore we need to proceed in a **backward** way, starting from the last time step and going backward in time. In particular we need to proceed in the following order

1. solve the subproblem parametrized by x_1;
2. compute the value function V_1;
3. solve the outer problem
$$\min_{u_0} \{ g(X_0, u_0) + V_1(f_0(X_0, u_0)) \}$$

1.2 Dynamic Programming

We can now write the general DP algorithm, which is a backward procedure to compute the value function and the optimal policy. We start from the last time step and we proceed backward in time until we reach the initial time step.
At the last stage (T) the subproblem is trivial, since there are no more decisions to be taken, therefore we have

$$V_T(x_T) = g_T(x_T)$$

At time $T-1$ we compute the value function as

$$\min_{u_{T-1}} \{ g_{T-1}(x_{T-1}, u_{T-1}) + V_T(f_{T-1}(x_{T-1}, u_{T-1})) \}$$

In particular notice that in the formulation above we know the value of $V_T(x_T)$, therefore we can compute the value function at time $T-1$ and so on until we reach the initial time step. So at this moment we have an optimal policy for the time step $T-1$, that means that we know how to compute the optimal control action at time $T-1$ given the state at time $T-1$. We can repeat this procedure until we reach the initial time step, therefore we have an optimal policy for all time steps.
At time $T-2$ we compute $V_{T-2}(x_{T-2})$ as

$$\min_{u_{T-2}} \{ g_{T-2}(x_{T-2}, u_{T-2}) + V_{T-1}(f_{T-2}(x_{T-2}, u_{T-2})) \}$$

Notice that this value function $V_{T-2}(x_{T-2})$ is a function of the state x_{T-2}, therefore we have to compute it $\forall x_{T-2} \in \mathcal{X}_{T-2}$, which is generally a very hard task.

Problem Decomposition

We have seen that the optimal control problem is decomposed into **stage problems** that we can solve via **backward induction** :

$$V_t(x_t) = \min_{u_t}\{g_t(x_t, u_t) + V_{t+1}(f_t(x_t, u_t))\} \tag{2}$$

And $V_T(x_T) = g_T(x_T)$

A reasonable question at this point would be, in which cases is this stage problem formulation simple to solve ? We can list three scenarios:

- Decision space $\tilde{\mathcal{U}}$ is convec , or maybe finite.
- If g_t is convex in u_t.
- If V_{t+1} evaluated at the future step f_t is convex in u_t.

In practice, a very common setup in which the assumptions above are satisfied is when we have a **quadratic cost** g_t and a **linear dynamics** f_t, which is the so called **LQR problem**. Another way is also to use approximations of the value function, in which case maybe not all the assumptions above are satisfied, but we can still solve the stage problem.

1.3 Optimal LQR

The LQR problem is a special case of the optimal control problem, in which we have a linear dynamics and a quadratic cost. In particular we have:

- Markovian update and linear dynamics

$$x_{t+1} = A_t x_t + B_t u_t$$

- Quadratic cost function:

$$\sum_{t=0}^{T-1} \left(x_t^\top \, Q_t \, x_t + u_t^\top \, R_t \, u_t \right) + x_T^\top \, S \, x_T \quad \text{with } Q, S \geq 0 \, , \, R > 0$$

- Value function (i.e. minimum cost to go starting from x at time t) :

$$V_t(x) = \min_{u_t, \ldots, u_{T-1}} \sum_{k=t}^{T-1} \left(x_k^\top \, Q_k \, x_k + u_k^\top \, R_k \, u_k \right) + x_T^\top \, S \, x_T$$

$$\begin{aligned} \text{subject to } x_{k+1} &= A_k x_k + B_k u_k \quad \forall k = t, \ldots, T-1 \\ x_t &= x \end{aligned}$$

We say that the optimal cost is $V_0(x_0)$.

1.4 Dynamic Programming for LQR

Let us now specialize the dynamic programming recursion to the Linear Quadratic Regulator (LQR) problem. Recall that the value function represents the optimal *cost-to-go*, i.e. the minimum cost achievable from a given state when acting optimally from that point onward.
A key observation in the LQR setting is that the terminal cost is quadratic in the state. Indeed, from the problem formulation we have

$$V_T(x) = x^\top S x,$$

which is a convex quadratic function since $S \succeq 0$.
The main idea of the LQR derivation is to show that the value function $V_t(x)$ is also quadratic and has the form of $V_t(x) = x^\top P_t\, x$ for some symmetric positive semidefinite matrix P_t.
We will then show that the matrices P_t can then be computed recursively by working *backward in time* starting from the terminal time T. Eventually the the optimal control input u_t will be computed as a solution of a convex quadratic optimization problem, which admits a closed-form solution.

Proof. To derive the recursion, consider the dynamic programming stage problem at time t where the value function is

$$V_t(x) = \min_u \Big(x^\top Q\, x + u^\top R\, u + V_{t+1}(Ax + Bu) \Big).$$

We now proceed by induction, assuming that $V_{t+1}(x) = x^\top P_{t+1} x$ for some positive semidefinite matrix P_{t+1}. Then $V_t(x)$ can be written as

$$\begin{aligned} V_t(x) &= x^\top Q x + \min_u \Big(u^\top R u + (Ax + Bu)^\top P_{t+1}(Ax + Bu) \Big) \\ &= x^\top Q x + \min_u \Big(u^\top R u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x + u^\top B^\top P_{t+1} B u \Big) \\ &= x^\top\, Q\, x + \min_u \Big(u^\top (R + B^\top P_{t+1} B) u + x^\top A^\top P_{t+1} A x + 2u^\top B^\top P_{t+1} A x \Big). \end{aligned}$$

We set the gradient of the expression inside the minimization with respect to u to zero:

$$(R + B^\top P_{t+1} B)u + B^\top P_{t+1} A x = 0$$

Yielding the optimal control input

$$\boxed{u^* = -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x}$$

But now we need to check that $V_t(x)$ is indeed quadratic in X. That means $V_t(x) = x^\top P_t x$ for some P_t and we have to find how to compute P_t. We can do that by plugging the optimal control input u^* back into the expression for $V_t(x)$:

$$\begin{aligned} V_t(x) &= x^\top Q x + (u^*)^\top R u^* + (Ax + Bu^*)^\top P_{t+1}(Ax + Bu^*) \\ &= x^\top Q x + (u^*)^\top R u^* + x^\top A^\top P_{t+1} A x + 2(u^*)^\top B^\top P_{t+1} A x + (u^*)^\top B^\top P_{t+1} B u^* \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top (R + B^\top P_{t+1} B) u^* + 2(u^*)^\top B^\top P_{t+1} A x \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + (u^*)^\top \big((R + B^\top P_{t+1} B) u^* + 2B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x \\ &\quad -\big((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x\big)^\top \big(-(R + B^\top P_{t+1} B)(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x + 2B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x + -\big((R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x\big)^\top \big(B^\top P_{t+1} A x \big) \\ &= x^\top Q x + x^\top A^\top P_{t+1} A x - x^\top A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A x \\ &= x^\top \Big(Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \Big) x \\ &= x^\top P_t x \end{aligned}$$

To complete the proof one should check that P_t is positive semidefinite. Here we homit the proof.

In cocnlusion we have been able to derive a closed-form expression for the optimal control input u^* for the LQR problem.Be aware that this solution is not valid anymore if the problem becomes constrained. □

We summerize here the main results:

Optimal LQR solution

Backward induction

$$u_t = \underbrace{-(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A}_{\Gamma_t} \, x_t$$

where

$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \quad \text{and } P_T = S$$

Forward integration from x_0:

$$x_{t+1} = Ax_t + Bu_t$$

So clearly one could compute this whole open-loop sequence $u_0, \dots, u_{T-1}$ in a offline computation. But what if the optimal control input $\mathbf{u}$ takes us to a trajectory different from the one we compouted ? In this case we can use the optimal policy Γ_t to compute the optimal control input at each time step, given the current state x_t. This is the main advantage of having a closed-form solution for the optimal control input, since we can use it in a feedback way.

We can do a quick comparison of the computational cost of using these equations in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**
  - $n \times n$ matrix multiplications to compute P_t for $t = T-1, \dots, 0$
  - $m \times m$ matrix inversions to compute Γ_t for $t = T-1, \dots, 0$
  - T iterations to compute P_t and Γ_t for $t = T-1, \dots, 0$
- **Online computation**
  - Storage of T $m \times n$ matrices Γ_t for $t = T-1, \dots, 0$
  - $m \times m$ matrix multiplications to compute u_t at each time step, given x_t

1.4.1 Infinite-Horizon LQR

In many practical control applications the system is required to operate over a very long time horizon. Examples include regulation problems where the controller must stabilize a system indefinitely, or tracking tasks where disturbances may occur continuously.

For these reasons it is natural to study the limit case where the horizon tends to infinity. The resulting problem is known as the **infinite-horizon Linear Quadratic Regulator (LQR)**.

Consider the linear time-invariant system

$$x_{t+1} = Ax_t + Bu_t, \qquad x_0 \text{ given}$$

and the infinite-horizon quadratic cost

$$\min_{u_0, u_1, \dots} \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + u_t^\top R u_t \right), \qquad Q \succeq 0, \; R \succ 0.$$

Compared to the finite-horizon case, the cost now accumulates indefinitely. Therefore a first important question concerns the **feasibility** of the problem.

Feasibility

If the pair (A, B) is **stabilizable**, then there exists a control input sequence that yields a finite value of the infinite-horizon cost.

Indeed, if the system is stabilizable there exists a feedback matrix K such that the closed-loop matrix $A + BK$ has all eigenvalues inside the unit circle. Consider the linear feedback policy

$$u_t = Kx_t.$$

The state then evolves according to

$$x_t = (A + BK)^t x_0.$$

Substituting this into the cost yields

$$V(x_0) = \sum_{t=0}^{\infty} \left(x_t^\top Q x_t + x_t^\top K^\top R K x_t \right).$$

Using the closed-loop dynamics we obtain

$$V(x_0) = x_0^\top \left(\sum_{t=0}^{\infty} (A + BK)^{t\top} (Q + K^\top R K)(A + BK)^t \right) x_0.$$

Since $A + BK$ is stable, the above summation behaves like a *matrix geometric series* and therefore converges. Consequently the cost is finite.

But how do we compute the optimal policy for the infinite-horizon LQR problem ? The answer is that we can still apply the dynamic programming recursion, but now the value function does not depend on time. The solution is given by the **Discrete Algebraic Riccati Equation (DARE)**, which is a fixed-point equation for the matrix P that defines the value function.

Algebraic Riccati Equation

The iteration

$$P_{t-1} = Q + A^\top P_t A - A^\top P_t B (R + B^\top P_t B)^{-1} B^\top P_t A \qquad P_0 = 0$$

converges, for $t \to -\infty$, to a unique positive semidefinite solution $P_\infty \succ 0$ of the algebraic Riccati equation

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A$$

We can show that then the optimal feedback control in the infinite-horizon case is given by

$$u_t = \underbrace{-(R + B^\top P B)^{-1} B^\top P A}_{F_\infty} x_t$$

and thus minimizes the infinite horizon cost and yields $V(X_0) = X_0^\top P X_0$.
We now show again a comparison of the computational cost of using the infinite-horizon solution in a feedback way vs computing the whole open-loop sequence. By fixing m as the size of the states and n as the size of the control input, we have the following computational costs:

- **Offline computation**

  - $n \times n$ matrix multiplications to compute P
  - $m \times m$ matrix inversions to compute F_∞
  - T iterations to compute P and F_∞ for $t = T-1, \dots, 0$
  - "Infinite iterations" to compute P for $t \to -\infty$[1]

- **Online computation**
  - Storage of a **single** $m \times n$ matrix F_∞. So less memory is required compared to the finite-horizon case, since we do not need to store T matrices Γ_t.
  - $m \times n$ matrix multiplications to compute u_t at each time step, given x_t

A natural question at this point is whether the optimal infinite-horizon feedback law is also *stabilizing*. Indeed, even if the infinite-horizon cost is well defined and the Riccati equation admits a solution, one would still like to know whether the resulting closed-loop dynamics

$$x_{t+1} = (A + BF_\infty)x_t$$

are asymptotically stable.

This is a meaningful question because the quadratic cost does *not* necessarily penalize all state directions. In particular, if some unstable state component is not "seen" by the cost, then the optimizer may have no incentive to stabilize it.

Illustrative example Consider the system

$$x_{t+1} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} x_t + u_t, \qquad Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Here both open-loop modes are unstable, since the eigenvalues of A are 2 and 3. However, the stage cost penalizes only the first state component x_1, while the second component x_2 does not appear in the state cost. Therefore, from the viewpoint of the objective function, an unstable behavior in the second state may go completely unnoticed unless it is indirectly forced to be controlled through the dynamics.

This already suggests an important principle:

> Unstable modes must be visible in the cost function, otherwise the optimal controller may fail to stabilize them.

To make this statement precise, write $Q = C^\top C$. Then the cost penalizes exactly the directions that are observable through the pair (C, A).

Stability of the optimal infinite-horizon LQR controller

Let $Q = C^\top C$. The optimal infinite-horizon feedback F_∞ stabilizes the system if and only if

1. the pair (A, B) is stabilizable;
2. the pair (C, A) has no unobservable unstable modes.

Equivalently, all unstable modes of A must be detectable through the state penalty Q.

The second condition is often stated by saying that the pair $(A, Q^{1/2})$ must be **detectable**. Detectability is weaker than observability: it does not require all modes to be observable, but only the unstable ones.

[1] P_∞ can be computed by solving the Algebraic Riccati Equation

Interpretation This theorem has a clear practical meaning. The matrix R penalizes the control effort, while Q determines which state directions the controller tries to regulate. If an unstable mode is not penalized by Q, then the optimization problem may prefer not to spend control effort on that mode, since doing so does not improve the objective. As a consequence, the optimal controller may exist but fail to stabilize the closed-loop system.

Hence, in practice:

- (A, B) stabilizable ensures that unstable modes can in principle be controlled;
- detectability of $(A, Q^{1/2})$ ensures that every unstable mode matters in the cost.

When both conditions hold, the stabilizing solution P of the algebraic Riccati equation yields the optimal feedback

$$u_t = F_\infty x_t, \qquad F_\infty = -(R + B^\top P B)^{-1} B^\top P A,$$

and the closed-loop matrix $A + BF_\infty$ has all eigenvalues inside the unit disk.

2 Model Predictive Control

2.1 Introduction

In the previous chapter we have seen an overview of the LQR problem and how to solve it using dynamic programming. In that formulation, we assumed that there were no constraints on the state and control input. However, in many real-world applications, constraints on the state and input must be explicitly taken into account.
In the presence of constraints, the problem cannot, in general, be solved in closed form via backward induction as in the unconstrained LQR case, since the value function is no longer quadratic.The resulting optimization problem becomes:

Constrained LQ finite horizon problem

$$\begin{aligned}\min_{u_0,\dots,u_{T-1},x_0,\dots,x_T} \quad & \sum_{t=0}^{T-1} \left(x_t^\top Q x_t + u_t^\top R u_t \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t, \quad t = 0, \dots, T-1 \\ & x_0 = x_0 \\ & x_t \in \mathcal{X}_t \\ & u_t \in \mathcal{U}_t\end{aligned}$$

Usually, the constraint sets are convex, e.g., polytopes.

One could solve this optimization problem in a **one-shot** fashion (i.e., open-loop strategy), obtaining an optimal input sequence $u_0, \dots, u_{T-1}$ for the given initial state x_0, instead of a state-feedback policy as in the unconstrained case (where $u_t = \Gamma_t x_t$).
However, if disturbances or model mismatch cause the system to deviate from the predicted trajectory, the previously computed control sequence is no longer optimal. As a result, the system will follow a different trajectory than expected (see Figures 1 and 2).

To address this issue, the optimal control problem must be re-solved using the updated state information. Since we assume a **Markovian system**, the current state contains all the information needed to compute an optimal control sequence for the future. This implies that the optimization problem must be solved repeatedly, at each time step, within one sampling interval.

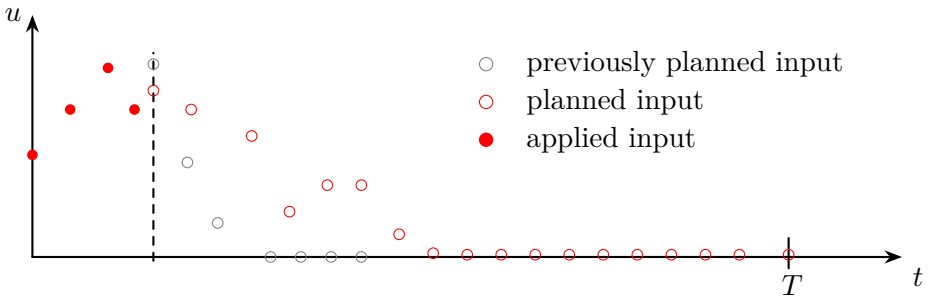

Figure 1: Receding-horizon update of the input sequence: previously planned inputs are shifted, a new optimal input sequence is computed based on the current state.

The idea that emerges from this discussion is to solve the optimization problem in a **receding horizon** fashion. At each time step, we solve a finite-horizon optimal control problem of length T, and apply only the first control input of the optimal sequence (see Figure 3).
At the next time step, the new state is measured, and the optimization problem is solved again

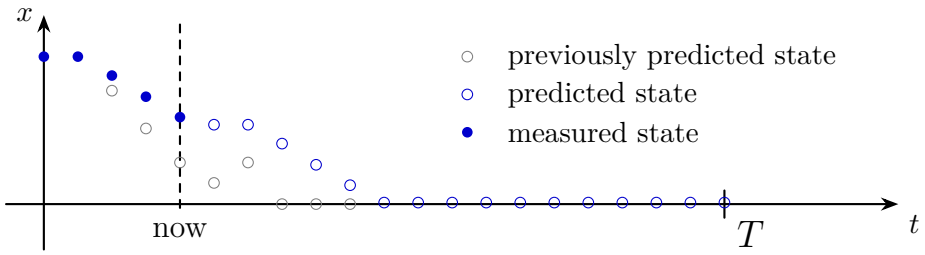

Figure 2: Receding-horizon update of the state trajectory: previously predicted states are corrected using the measured state at the current time, and a new optimal state trajectory is predicted over the horizon.

with the updated initial condition. If the system is time-invariant, the optimization problem remains the same at each time step, except for the initial state. The corresponding predicted state evolution over the horizon is illustrated in Figure 4.

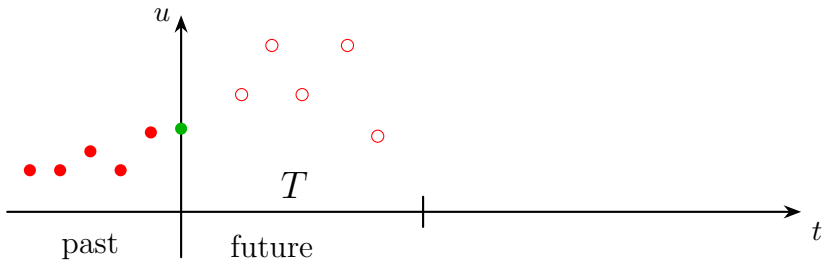

Figure 3: Finite-horizon input sequence in receding horizon control: at the current time, an optimal sequence of future inputs over a horizon of length T is computed, while past inputs are fixed.

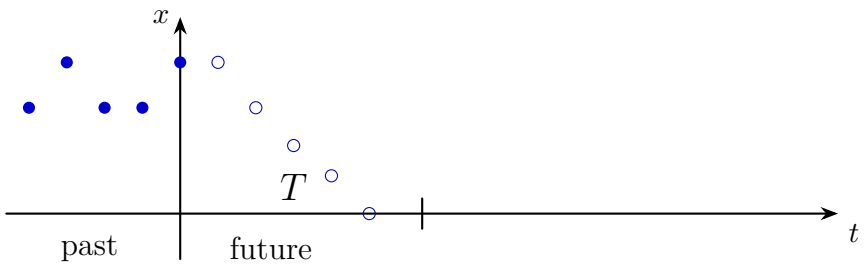

Figure 4: Finite-horizon state prediction in receding horizon control: starting from the current state, future states are predicted over a horizon of length T using the planned input sequence.

Thus the following scheme defines the receding horizon control strategy, also known as **Model Predictive Control (MPC)**:

1. At time t, measure the current state x_t.
2. Solve the finite-horizon optimal control problem with initial state x_t to obtain the optimal input sequence $u_t, u_{t+1}, \ldots, u_{t+T-1}$.
3. Apply the first control input u_t to the system.
4. Increment time $t \leftarrow t + 1$ and repeat from step 1.

The resulting **open-loop optimal control problem** corresponds to a *parametric optimization problem* parametrized in the initial state x.

Parametric optimization problem

$u^*(x)$ is determined by[a]

$$\begin{aligned} \min_{\mathbf{u},\mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k \\ & x_0 = x \\ & x_k \in \mathcal{X}_k \\ & u_k \in \mathcal{U}_k \end{aligned}$$

[a] Notation: k is the index for the optimization problem, while t is the index for the time steps of the system.

The parametric nature of the problem immediately raises the question of whether the resulting control strategy is *feedforward* or *feedback*. Although the optimization problem computes an open-loop input sequence, the receding horizon implementation **introduces feedback implicitly**. Indeed, at each time step the optimization problem is solved using the current state x_t, and only the first input of the optimal sequence is applied. Therefore, the implemented control law can be written as

$$u_t = u^*(x_t),$$

which defines a **state-feedback control law**. One can have a visual intuition of this feedback structure by looking at the block diagram in Figure 5.

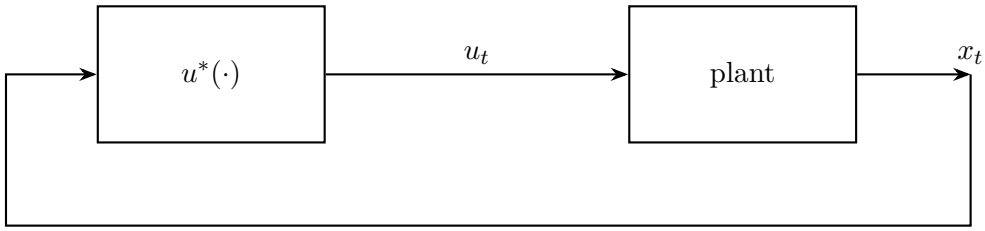

Figure 5: Block diagram of the MPC control scheme

We can establish some properties about this optimization problem:

- Under **continuity** assumptions on the cost and the dynamics, the minimum cost is a continuous function of the initial state x.
- Under **compactness** assumptions on the constraints, the parametric optimization problem admits a solution for all x in the feasible set.
- The map $x \mapsto u^*(x)$ is not necessarily continuous for a general cost function

In the case of linear dynamics, quadratic cost, and convex constraints, the optimization problem is a **convex quadratic program (QP)** parametrized in the initial state x. In this case, the map $x \mapsto u^*(x)$ is piecewise affine and continuous. However, in general, the control law induced by MPC can be nonlinear and non-smooth.

MPC control law

As a consequence, MPC can be interpreted as a **memory-less, nonlinear, time-invariant feedback control law**. The nonlinearity arises from the presence of constraints, even when the system dynamics are linear and the cost is quadratic.

A natural question is whether MPC can achieve *perfect tracking*, i.e., zero steady-state error. In general, this is not guaranteed without additional design choices (e.g., terminal ingredients or disturbance modeling), since the controller repeatedly solves a finite-horizon problem and does not explicitly enforce asymptotic properties beyond the prediction horizon. To better understand the relation between MPC and the finite-horizon optimal control problem, consider the following example:

Example

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{t=0}^{T-1} a^t |u_t| + |x_T|^2, \quad |a| < 1 \\ \text{s.t.} \quad & x_{t+1} = x_t + u_t. \end{aligned}$$

Two different questions can be asked:

1. What is the optimal open-loop solution $(\mathbf{x}^*, \mathbf{u}^*)$?
2. What is the MPC feedback law $x \mapsto u^*(x)$ obtained by solving the problem in a receding horizon fashion?

To solve this we observe that the cost is not quadratic and that the cost of applying inouts is discounted over time. Furthermore the state is penalized only at the end of the horizon.
Thus the optimal solution is to simply wait for the last time step and then apply $u_T = -x_T$, which yields a cost of $|x_0|^2$.
In contrast, the MPC feedback law is given by the control law $x \mapsto u^*(x) = 0$. This is because at each time step the optimization problem is solved with a shifted horizon, and the optimal input sequence is always to wait until the end of the horizon and then apply $u_{t+T} = -x_{t+T}$. Therefore, at each time step, the first input of the optimal sequence is zero.
A key observation is that the **optimal trajectory** associated with the finite-horizon problem is, in general, **never realized in closed loop**, even in the absence of disturbances or model mismatch. This is because at each time step the optimization problem is re-solved with a shifted horizon, leading to a different optimal sequence than the one computed previously. ■

Consider now the **unconstrained linear-quadratic** problem:

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{t=0}^{T-1} \left(x_t^\top Q x_t + u_t^\top R u_t \right) + x_T^\top S x_T, \quad Q, S \succeq 0,\ R \succ 0 \\ \text{s.t.} \quad & x_{t+1} = A x_t + B u_t. \end{aligned}$$

Finite-time LQR yields a time-varying linear feedback law

$$\begin{aligned} u_t &= \Gamma_t x_t \\ \Gamma_t &= -(R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \end{aligned}$$

where the matrices P_t are computed backward in time. The control gain explicitly depends on time through P_{t+1}.
In contrast, **MPC** repeatedly solves the same finite-horizon problem at each time step. As a result, the applied control law is

$$u_t = \Gamma_0 x_t,$$

where Γ_0 is the gain associated with the first step of the horizon. Hence, MPC induces a **linear time-invariant** feedback law.
More explicitly, at time $t = 0$ both approaches yield the same control input:

$$u_0^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_0 = u_0^{\text{LQR}}.$$

However, at the next time step a discrepancy appears. Finite-time LQR applies

$$u_1^{\text{LQR}} = -(R + B^\top P_2 B)^{-1} B^\top P_2 A x_1,$$

while MPC recomputes the problem and applies again

$$u_1^{\text{MPC}} = -(R + B^\top P_1 B)^{-1} B^\top P_1 A x_1.$$

Therefore, even in the unconstrained case, **MPC does not reproduce the finite-horizon optimal control sequence beyond the first step**.

2.2 Implementation of MPC

The implementation of MPC requires solving an optimization problem at each time step. We show two main approaches to implement MPC:

1. Compute the **function** $u^*(\cdot)$ **offline**, and then evaluate it online at each time step.
   - It is extremely complex to solve with exception of very few cases
   - Abundant offline computational resources are required
   - Large memory is required to store the function $u^*(\cdot)$
2. Compute the **value** of the function $u^*(\cdot)$ **online** at each time step by solving the optimization problem.
   - It is often a tractable optimization problem.
   - It has limited time to perform computation online.

2.2.1 Explicit MPC

Explicit MPC corresponds to the **first implementation strategy**: instead of solving the optimization problem online at each sampling instant, we try to compute the feedback law $x \mapsto u^*(x)$ **offline**.
We have already seen one important case. In the **unconstrained linear MPC** case, the optimizer is simply the first element of the finite-horizon LQR solution. Therefore, the applied input can be written as

$$u_t = u^*(x_t) = \Gamma_0 x_t,$$

so the control law is linear.
The natural question is what happens when **constraints are introduced**. Consider the standard constrained linear MPC problem

Linear MPC with linear constraints

$$\begin{aligned}
\min_{\mathbf{u},\mathbf{x}} \quad & \sum_{k=0}^{T-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_T^\top S x_T \\
\text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \qquad k = 0, \dots, T-1, \\
& x_0 = x, \\
& x_k \in \mathcal{X}, \qquad \mathcal{X} = \{x \in \mathbb{R}^n : Cx \le d\}, \\
& u_k \in \mathcal{U}, \qquad \mathcal{U} = \{u \in \mathbb{R}^m : Eu \le f\},
\end{aligned}$$

with $Q, S \succeq 0, \qquad R \succ 0.$
This is the most common MPC formulation: linear dynamics, quadratic cost, and polyhedral state/input constraints.

This problem is a **convex quadratic program** parametrized by the initial state x. In general, the optimizer is no longer globally linear. However, due to the quadratic cost and linear constraints, it still has a very specific structure: the explicit MPC law turns out to be **piecewise affine**. The idea is best understood through a very simple example.

Example Consider a scalar integrator with horizon $T = 1$:

$$\begin{aligned} \min_{u_0, x_1} \quad & u_0^2 + x_1^2 \\ \text{s.t.} \quad & x_1 = x_0 + u_0, \\ & x_1 \leq 1. \end{aligned}$$

This example is useful because:

- it is a quadratic problem with a linear constraint,
- it is parametrized by the current state x_0,
- the equality constraint allows us to eliminate the variable x_1,
- the problem is convex, hence the KKT conditions are necessary and sufficient.

Using the model equation $x_1 = x_0 + u_0$, we can eliminate x_1 and rewrite the problem as

$$\min_{u_0} f(u_0) \qquad \text{s.t.} \qquad g(u_0) \leq 0,$$

where

$$\begin{aligned} f(u_0) &= u_0^2 + (x_0 + u_0)^2, \\ g(u_0) &= x_0 + u_0 - 1. \end{aligned}$$

The derivatives are

$$\begin{aligned} \nabla f(u_0) &= 4u_0 + 2x_0, \\ \nabla g(u_0) &= 1. \end{aligned}$$

KKT conditions for one inequality constraint

For a problem of the form

$$\min_{x} f(x) \qquad \text{s.t.} \qquad g(x) \leq 0,$$

the KKT conditions are

$$\begin{aligned} \nabla f(x) + \mu \nabla g(x) = 0, \qquad & \mu \geq 0, \\ g(x) \leq 0, & \\ \mu\, g(x) = 0. & \end{aligned}$$

The last relation is the *complementary slackness* condition.

Applying the KKT conditions to the problem gives

$$\begin{aligned} 4u_0 + 2x_0 + \mu = 0, \qquad & \mu \geq 0, \\ x_0 + u_0 - 1 \leq 0, & \\ \mu(x_0 + u_0 - 1) = 0. & \end{aligned}$$

The complementary slackness condition implies that two cases are possible.

Case 1: inactive constraint. Assume first that the inequality is inactive, so

$$\mu = 0.$$

Then stationarity becomes

$$4u_0 + 2x_0 = 0,$$

hence

$$u_0 = -\frac{x_0}{2}.$$

To check whether this candidate is feasible, we substitute it into the constraint:

$$x_0 + u_0 - 1 = x_0 - \frac{x_0}{2} - 1 = \frac{x_0}{2} - 1.$$

Therefore feasibility requires

$$\frac{x_0}{2} - 1 \leq 0 \qquad \Longleftrightarrow \qquad x_0 \leq 2.$$

So, for all initial states satisfying $x_0 \leq 2$, the optimal input is

$$u_0^* = -\frac{x_0}{2}.$$

Case 2: active constraint. Assume now that the inequality is active, so

$$x_0 + u_0 - 1 = 0.$$

Then

$$u_0 = 1 - x_0.$$

Substituting this into the stationarity condition,

$$4u_0 + 2x_0 + \mu = 0,$$

we obtain

$$\mu = -4u_0 - 2x_0 = -4(1 - x_0) - 2x_0 = 2x_0 - 4.$$

Since $\mu \geq 0$, this case is valid only if

$$2x_0 - 4 \geq 0 \qquad \Longleftrightarrow \qquad x_0 \geq 2.$$

Hence, for all $x_0 \geq 2$, the optimal input is

$$u_0^* = 1 - x_0.$$

Combining the two cases, the explicit solution is

Explicit solution of the toy MPC problem

$$u_0^*(x_0) = \begin{cases} -\dfrac{x_0}{2}, & x_0 \leq 2, \\ 1 - x_0, & x_0 \geq 2. \end{cases}$$

This example shows the main structural property of explicit MPC:

- the control law is **continuous**,
- the control law is **piecewise affine**,
- each affine expression is valid in a region where the same set of constraints is active.

The reason is the following. Once the active constraints are known, the KKT conditions reduce to a linear system in the decision variables and the multipliers. Solving that linear system gives an optimizer that depends *affinely* on the parameter x. When the active set changes, a different affine expression is obtained. Therefore, the state space is partitioned into regions, and on each region the controller is affine.

Although the explicit solution is conceptually appealing, it becomes difficult to use in large problems. In practice:

- there may be tens or hundreds of constraints,
- the number of regions may become very large,
- the offline computation required to determine all regions can be very long,
- even online, one must determine in which region the current state lies before evaluating the affine law.

As a consequence, explicit MPC is mainly **attractive when the system dimension is small and the sampling time is so short that solving a quadratic program online would be too expensive**. For larger systems, the most common approach is instead to solve the MPC optimization problem online at each sampling instant.

2.3 Stability

A useful way to think about MPC is as a *feedback design tool*. Once the prediction horizon T and the weights Q, R, S are chosen, the receding-horizon implementation induces a control law of the form

$$u_t = u^*(x_t).$$

Even though the optimization problem computes an open-loop input sequence, only the first input is applied and the problem is solved again at the next time step. Therefore, the implemented controller is a **static, time-invariant, nonlinear feedback law**. The nonlinearity does not necessarily come from the dynamics; it already appears because of the repeated constrained optimization.
From this perspective, the main question is no longer only how to solve the optimization problem, but **whether the resulting closed-loop system is stable**. There are two conceptually different ways to approach this question. The first is an *analysis* point of view: one tries to compute the control law $u^*(\cdot)$ explicitly and then study the stability of the closed-loop dynamics. In general, this is difficult, because the MPC law can be piecewise affine or more generally nonlinear and non-smooth. The second is a *design* point of view: instead of computing the feedback law explicitly, one looks for **conditions on the design parameters T, Q, R, S that guarantee closed-loop stability directly**. This is the more useful viewpoint in practice.

To study stability, the standard tool is Lyapunov analysis. Consider a discrete-time system

$$x_{t+1} = f(x_t),$$

and suppose that the origin $x = 0$ is an equilibrium, that is, $f(0) = 0$.

Stable equilibrium

The equilibrium $x = 0$ is said to be **stable** if

$$\forall \varepsilon > 0,\ \exists \delta > 0 \text{ such that } \|x_0\| < \delta \implies \|x_t\| < \varepsilon \quad \forall t \geq 0.$$

In words, if the initial condition is sufficiently close to the equilibrium, then the entire trajectory remains close to it for all future times.

Stability alone only guarantees that trajectories do not move far away from the equilibrium. A **stronger notion is asymptotic stability**, which additionally requires convergence to the equilibrium.

Asymptotically stable equilibrium

The equilibrium $x = 0$ is **asymptotically stable** if it is stable and

$$\lim_{t\to\infty} \|x_t\| = 0.$$

Hence, trajectories starting sufficiently close to the origin remain close to it and eventually converge to it.

The classical criterion to prove asymptotic stability is based on a Lyapunov function.

Lyapunov theorem for discrete-time systems

Assume there exists a real-valued function $W : \mathbb{R}^n \to \mathbb{R}$ such that

$$\begin{aligned} W(0) &= 0, \\ W(x) &> 0 \qquad \forall x \neq 0, \\ W(f(x)) &< W(x) \qquad \forall x \neq 0. \end{aligned}$$

Then the equilibrium $x = 0$ is asymptotically stable.

The idea is simple: W plays the role of an **energy-like quantity**. It is zero only at the equilibrium, positive everywhere else, and strictly decreases along the closed-loop trajectories. Therefore, the system is forced to move toward the origin.

This Lyapunov viewpoint explains why stability is relatively transparent for *infinite-horizon* MPC. Suppose that, at time t, the controller computes an input sequence that minimizes the infinite-horizon cost

$$V_t^\infty(x, \mathbf{u}) = \sum_{k=t}^{\infty} g_k(x_k, u_k),$$

subject to the system dynamics. If the minimum is attained, it is natural to define the optimal value function

$$W(x) = \min_{\mathbf{u}} V_t^\infty(x, \mathbf{u}).$$

Now comes the key observation. By Bellman's principle of optimality, if an input sequence is optimal from time t, then its tail is optimal from time $t + 1$ onward. In other words, after applying the first optimal input $u^*(x_t)$ and reaching the next state x_{t+1}, the remaining part of the optimal sequence is still optimal for the optimization problem starting at $t + 1$. As a consequence, along the closed-loop trajectory one obtains

$$W(x_{t+1}) = W(x_t) - g_t(x_t, u^*(x_t)).$$

Therefore, if the stage cost satisfies suitable positivity properties, for example if

$$g_t(x_t, u_t) > 0 \qquad \text{for all } x_t \neq 0,$$

then

$$W(x_{t+1}) < W(x_t) \qquad \text{for all } x_t \neq 0.$$

This means that the **optimal value function is a natural Lyapunov candidate** for the closed-loop system. Under reasonable assumptions, it follows that the origin is asymptotically

stable.

This argument is very elegant, but it also hides some subtleties. First, an infinite-horizon MPC problem is infinite-dimensional, so from a computational viewpoint it is not directly implementable as stated. Second, one is implicitly assuming that the global minimum exists and is attained. Third, the stage cost must satisfy suitable properties so that the value function is positive definite and decreases strictly along the closed loop. These requirements are analogous in spirit to the assumptions encountered in infinite-horizon LQR.

The situation changes significantly for *finite-horizon* MPC. In that case, the same Lyapunov argument does not apply automatically. At time t, the controller minimizes a cost over the interval $[t, t+T]$. At time $t+1$, however, the optimization problem is not simply the tail of the previous one: the first stage cost has disappeared, but a new term has entered at the end of the horizon. Therefore, the controller at time $t+1$ is minimizing a *different* objective. As a result, the optimizer changes, and the **finite-horizon value function is not automatically decreasing along the closed-loop trajectory**.
This is the fundamental reason why closed-loop stability is immediate for infinite-horizon optimal control, but not for finite-horizon receding-horizon MPC. **For finite-horizon MPC, stability must be enforced through additional design choices**. This will be the starting point for the next part of the discussion.

A classical way to **recover a Lyapunov argument** for finite-horizon MPC is to modify the optimization problem by imposing a **terminal constraint**. The simplest choice is the *zero terminal constraint*, namely

$$x_{t+T} = 0.$$

The corresponding finite-horizon optimal control problem becomes

Finite-horizon MPC with zero terminal constraint

Given the current state x_t, define

$$\begin{aligned} W_T(x_t) = \min_{\mathbf{x},\mathbf{u}} \quad & \sum_{k=t}^{t+T-1} g_k(x_k, u_k) \\ \text{s.t.} \quad & x_{k+1} = f(x_k, u_k), \qquad k = t, \dots, t+T-1, \\ & x_t \text{ given}, \\ & x_{t+T} = 0. \end{aligned}$$

The intuition is very simple. By forcing the predicted state to reach the origin at the end of the horizon, we make the finite-horizon problem look more similar to the infinite-horizon one. In particular, once the terminal state is the equilibrium, the tail of the optimal trajectory can be completed in a natural way by keeping the system at the origin with zero input, provided that

$$f(0,0) = 0.$$

This terminal condition restores the key mechanism that was missing in the plain finite-horizon case. Indeed, suppose that at time t the optimal solution of the MPC problem is

$$\{\tilde{u}_t, \tilde{u}_{t+1}, \dots, \tilde{u}_{t+T-1}\},$$

with associated state trajectory

$$\{\tilde{x}_t, \tilde{x}_{t+1}, \dots, \tilde{x}_{t+T}\},$$

where by construction

$$\tilde{x}_{t+T} = 0.$$

After applying the first control input $\tilde{u}_t$, the system reaches the state

$$x_{t+1} = \tilde{x}_{t+1}.$$

At time $t+1$, a natural feasible candidate input sequence is then obtained by taking the *tail* of the previous optimizer and appending the zero input:

$$\{\tilde{u}_{t+1}, \tilde{u}_{t+2}, \dots, \tilde{u}_{t+T-1}, 0\}.$$

This is feasible because the previous optimal trajectory already satisfies $\tilde{x}_{t+T} = 0$, and the appended input $u = 0$ keeps the system at the equilibrium.
Therefore, the optimal cost at time $t+1$ can be upper bounded by the cost of this feasible candidate. This gives

$$\begin{aligned} W_T(x_{t+1}) &\leq \sum_{k=t+1}^{t+T} g_k(x_k, u_k) \\ &= \sum_{k=t}^{t+T-1} g_k(x_k, u_k) - g_t(x_t, u_t) + g_{t+T}(x_{t+T}, u_{t+T}). \end{aligned}$$

Since $x_{t+T} = 0$ and we append $u_{t+T} = 0$, the last term vanishes under the standard assumption that the stage cost is zero at the equilibrium:

$$g_{t+T}(0, 0) = 0.$$

Hence,

$$W_T(x_{t+1}) \leq W_T(x_t) - g_t(x_t, u_t).$$

If, in addition, the stage cost is positive away from the origin, then for every $x_t \neq 0$ we obtain

$$W_T(x_{t+1}) < W_T(x_t).$$

This shows that the optimal value function W_T decreases along the closed-loop trajectories, and therefore it becomes a Lyapunov candidate for the closed-loop system.

Asymptotic stability with zero terminal constraint

Under the usual assumptions of feasibility, attainment of the minimum, $f(0, 0) = 0$, and positivity of the stage cost, the equilibrium $x = 0$ is asymptotically stable for the closed-loop system generated by finite-horizon MPC with terminal constraint

$$x_{t+T} = 0.$$

The proof is essentially the shifted-sequence argument above. The terminal constraint ensures that the optimal trajectory at time t can be transformed into a feasible candidate at time $t+1$ simply by removing the first input and appending the zero input at the end. This gives a monotonic decrease of the value function, exactly as required by Lyapunov theory. We report here a more formal proof for completeness.

Proof. Let

$$W(x_t) = \min_{\mathbf{u},\mathbf{x}} \sum_{k=t}^{t+T-1} g(x_k, u_k)$$

subject to

$$x_{k+1} = f(x_k, u_k), \qquad x_t \text{ given}, \qquad x_{t+T} = 0.$$

We want to compare the optimal cost at time $t+1$ with the one at time t. By definition,

$$\begin{aligned} W(x_{t+1}) &= \min \sum_{k=t+1}^{t+T} g(x_k, u_k) \\ &= \min \left(\sum_{k=t}^{t+T-1} g(x_k, u_k) - g(x_t, u_t) + g(x_{t+T}, u_{t+T}) \right). \end{aligned}$$

By definition of the minimum, this quantity is less than or equal to the same expression evaluated at any feasible input sequence. In particular, let

$$\tilde{\mathbf{u}} = \arg\min_{\mathbf{u},\mathbf{x}} \sum_{k=t}^{t+T-1} g(x_k, u_k)$$

be the optimizer of the MPC problem at time t, with associated state trajectory

$$\tilde{x}_{t+1}, \tilde{x}_{t+2}, \ldots, \tilde{x}_{t+T}, \qquad \tilde{x}_{t+T} = 0.$$

Now construct a feasible candidate input sequence for the problem at time $t+1$ by taking the tail of the previous optimal sequence and appending the input

$$u_{t+T} = 0.$$

Since $\tilde{x}_{t+T} = 0$ and $f(0,0) = 0$, this implies

$$x_{t+T+1} = 0,$$

so the terminal constraint is satisfied also at time $t+1$.
Evaluating the cost at this feasible candidate yields

$$\begin{aligned} W(x_{t+1}) &\le \sum_{k=t+1}^{t+T} g(\tilde{x}_k, \tilde{u}_k) \\ &= \sum_{k=t}^{t+T-1} g(\tilde{x}_k, \tilde{u}_k) - g(\tilde{x}_t, \tilde{u}_t) + g(\tilde{x}_{t+T}, u_{t+T}). \end{aligned}$$

Because $\tilde{x}_{t+T} = 0$ and $u_{t+T} = 0$, and assuming $g(0,0) = 0$, we obtain

$$\begin{aligned} W(x_{t+1}) &\le \sum_{k=t}^{t+T-1} g(\tilde{x}_k, \tilde{u}_k) - g(\tilde{x}_t, \tilde{u}_t) \\ &= W(x_t) - g(\tilde{x}_t, \tilde{u}_t) \\ &\le W(x_t). \end{aligned}$$

Hence the optimal value function is non-increasing along the closed-loop trajectories. If moreover

$$g(x, u) > 0 \qquad \forall x \neq 0,$$

then

$$W(x_{t+1}) < W(x_t) \qquad \forall x_t \neq 0,$$

so W is a Lyapunov function for the closed-loop system. Therefore, the equilibrium $x = 0$ is asymptotically stable. □

The zero terminal constraint is conceptually very appealing because it gives a clean and direct stability argument. However, it can also be **quite restrictive**. Requiring the predicted state to reach the origin exactly in T steps may be impossible for some initial conditions, especially when the horizon is short or constraints are tight. In other words, **stability can be obtained, but possibly at the price of a reduced feasible set**.

The zero terminal constraint is only one possible way to guarantee stability of finite-horizon MPC. More generally, several families of sufficient conditions can be used:

- choosing the prediction horizon T large enough;
- imposing a zero terminal constraint;
- imposing a suitable **terminal region** constraint for the state;
- combining terminal ingredients, such as terminal constraints and terminal penalties.

The common idea behind all these approaches is to recover, in one form or another, a Lyapunov-like decrease of the optimal cost along the closed-loop trajectories. The precise technical construction changes from one method to another, but the objective is always the same: to ensure that the receding-horizon controller does not merely optimize over the next T steps, but also induces a long-term stabilizing behavior.

When discussing MPC stability results, a few important subtleties should always be kept in mind.
First, throughout the analysis we implicitly assume that the MPC optimization problem is **feasible**. This is a nontrivial assumption. In particular, with a zero terminal constraint the problem may easily become infeasible, because not every state can be driven exactly to the origin in T steps while satisfying the constraints.
Second, **future feasibility may depend on the control action selected at the current time**. Thus, it is not enough to have feasibility only at the initial time: one would like feasibility to be preserved as the horizon recedes. This issue is often referred to as *recursive feasibility*, and it is closely connected to the choice of terminal ingredients.
Third, all these arguments assume that the **global minimum** of the optimization problem is found. This is automatic in the convex quadratic MPC problems discussed so far, but it becomes more delicate in more general nonlinear or nonconvex settings.
For these reasons, stability of MPC is not only a matter of optimization, but also of careful controller design. The horizon length, the stage cost, and the terminal ingredients must be selected jointly so that the resulting receding-horizon law is both feasible and stabilizing.

2.4 Disturbance rejection

So far, the MPC design has been discussed mainly in nominal conditions, that is, under the assumption that the model is exact and that no unknown perturbation acts on the system. In practice, however, disturbances are unavoidable. Depending on the application, the controller may be required to achieve different levels of robustness.

Disturbance rejection specifications

In an MPC problem, robustness against disturbances may refer to different objectives:

1. **rejection at steady state**, that is, **zero offset**;
2. **rejection during the transient**, namely a good response while the disturbance is acting and being compensated;
3. **guaranteed constraint satisfaction** despite disturbances.

These are related but distinct goals. A controller may remove the steady-state error and still react poorly during the transient. Conversely, a controller may have a reasonable transient behavior but fail to guarantee that state and input constraints are respected under uncertainty. In this subsection we focus on the first two aspects: zero steady-state offset and transient disturbance rejection.

2.4.1 Zero steady-state offset and incremental MPC

It is useful to remember that MPC is, in the end, a nonlinear static feedback law of the form

$$u_t = u^*(x_t).$$

This observation immediately suggests a **possible weakness**. If the plant requires a *nonzero* constant input in order to keep the state at the desired equilibrium, then a standard MPC formulation centered at the origin may fail to achieve

$$x_t \to 0.$$

In other words, a constant disturbance or a model mismatch may produce a **steady-state offset**. This issue is especially visible in the presence of:

- input disturbances,
- process noise,
- multiplicative uncertainty.

Moreover, trying to compensate this effect by choosing very small input weights may lead to poor numerical conditioning of the optimization problem. A standard remedy is to reformulate the controller in **incremental form**. Instead of using the input u_t directly as the manipulated variable, one introduces the input increment

$$\Delta u_t$$

and imposes the update law

$$u_{t+1} = u_t + \Delta u_t.$$

This is equivalent to augmenting the plant with an input integrator. If the original nominal model is

$$x_{t+1} = Ax_t + Bu_t,$$

then the augmented model becomes

$$\begin{bmatrix} x_{t+1} \\ u_{t+1} \end{bmatrix} = \begin{bmatrix} A & B \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ u_t \end{bmatrix} + \begin{bmatrix} 0 \\ I \end{bmatrix} \Delta u_t.$$

The optimization problem is then written in terms of Δu_t rather than u_t. This has two important consequences. First, the controller acquires an **integral action**, which is exactly what is needed to remove steady-state offsets caused by constant disturbances. Second, the input penalty in the stage cost now penalizes changes in the control signal, that is, it penalizes Δu_t rather than the absolute value of u_t. As a consequence, the controller naturally prefers smooth input profiles. Hence, the incremental formulation can be interpreted as an **integral nonlinear feedback law**: the feedback structure is still given by MPC, but the plant seen by the optimizer includes an integrator on the input channel. This is the basic mechanism used in MPC to obtain zero steady-state offset against constant disturbances.

2.4.2 Disturbance rejection during transients

Removing the steady-state error is often not enough. In many applications, the controller should also react well *while* the disturbance is affecting the system. A common strategy in MPC is based on two steps:

1. estimate the disturbance using the available measurements and the plant model;
2. correct the steady-state target using the estimated disturbance.

The idea is the following. The nominal prediction model alone cannot explain the measured plant behavior when a disturbance is present. Therefore, an additional disturbance variable is introduced and estimated online. The estimate is then used to shift the equilibrium that the MPC controller should track.
Unless better prior information is available, a very common choice is to model the disturbance as **constant**:

$$d_{t+1} = d_t, \qquad d_t \in \mathbb{R}^{n_d}, \qquad n_d \leq n.$$

This means that the unknown perturbation is assumed to vary slowly compared with the system dynamics, so that over the time scale of interest it can be regarded as approximately constant.

Step 1: Disturbance estimation Assume that the plant is described by the model

$$x_{t+1} = Ax_t + Bu_t + B_d d_t,$$

where d_t is an unknown disturbance entering through the matrix B_d. Since d_t is not measured, it must be estimated from the observed evolution of the state.
A simple and effective choice is a Luenberger-type disturbance observer. One first predicts the next state using the current state, the applied input, and the current disturbance estimate:

$$\hat{x}_{t+1} = Ax_t + Bu_t + B_d \hat{d}_t.$$

Then the disturbance estimate is corrected using the prediction error:

$$\hat{d}_{t+1} = \hat{d}_t + L(x_{t+1} - \hat{x}_{t+1}),$$

where L is an observer gain chosen so that

$$I - LB_d$$

is stable.
The logic is very intuitive. If the measured next state differs from the nominal prediction, the mismatch is interpreted as the effect of an unmodeled disturbance. The observer then updates $\hat{d}_t$ in order to make future predictions consistent with the measurements. In this way, the estimate gradually converges to the constant disturbance that is acting on the plant.

Step 2: correction of the steady-state target Once the disturbance has been estimated, the next step is to compute which steady state should be tracked by the controller. If the disturbance were absent, the desired equilibrium for a regulation problem would usually be the origin. However, when a constant disturbance is present, the true equilibrium generally shifts. Therefore, instead of forcing MPC to regulate the state to zero, we first compute a **disturbance-consistent steady-state target** (x_s, u_s).
Under the constant disturbance model, the current estimate $\hat{d}_t$ is our best approximation of the steady-state disturbance. The corresponding steady-state target can be obtained by solving

Steady-state target selection

$$\begin{aligned} \min_{x_s,u_s} \quad & \|x_s\|_{Q_s}^2 + \|u_s\|_{R_s}^2 \\ \text{s.t.} \quad & \begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = B_d \hat{d}_t, \\ & x_s \in \mathcal{X}, \\ & u_s \in \mathcal{U}. \end{aligned}$$

The equality constraint is simply the steady-state balance equation for the disturbed system. Indeed, at equilibrium we must have

$$x_s = Ax_s + Bu_s + B_d \hat{d}_t,$$

which is equivalent to the constraint above. The optimization criterion is used to select, among all admissible equilibria compatible with the disturbance estimate, the one that is preferred according to the weights Q_s and R_s.
Once (x_s, u_s) has been computed, the MPC controller is no longer asked to regulate the plant to the nominal origin. Instead, it regulates the *deviations*

$$\tilde{x}_t = x_t - x_s, \qquad \tilde{u}_t = u_t - u_s$$

toward zero. In this way, the controller tracks the correct equilibrium associated with the current disturbance estimate. As the estimate $\hat{d}_t$ evolves, the target (x_s, u_s) is updated accordingly. This is the basic mechanism that improves disturbance rejection during transients.

2.4.3 Advanced disturbance models

The constant-disturbance model is often a good default choice, but it is not the only possibility. A more general principle is summarized by the following statement.

Good regulator theorem

"Every good regulator of a system must be a model of that system."

The message is that a controller can reject well only those classes of disturbances that are represented, explicitly or implicitly, in its internal model. Therefore, if better prior knowledge about the disturbance is available, that knowledge should be used.

For instance, the disturbance may be:

- constant,
- ramp-like,
- periodic.

This is precisely the viewpoint of the **internal model principle**: the closed-loop system can reject disturbances belonging to the class that has been modeled in the controller design.
If the disturbance has its own linear dynamics

$$d_{t+1} = A_d d_t,$$

then the plant can be augmented as

$$\begin{bmatrix} x_{t+1} \\ d_{t+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & A_d \end{bmatrix} \begin{bmatrix} x_t \\ d_t \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_t.$$

In this form, the disturbance is treated as an additional dynamical state. This model can then be used in two ways:

- inside the disturbance estimator;
- directly inside the MPC prediction model, instead of merely shifting the steady-state target.

This framework includes the constant-disturbance model as a special case, obtained by taking

$$A_d = I.$$

More generally, the choice of A_d determines which class of disturbances the controller is able to reject. In the spirit of the internal model principle, one integrator is associated with rejection of step-like disturbances, while more complex models are needed for ramps or periodic signals.

TO FINISH

3 Economic MPC

In the previous chapters we have always implicitly assumed that we are regulating the steady state $x_S = 0$, $u_S = 0$.
However, in many applications we are interested in regulating to a nonzero setpoint, or even tracking a time-varying reference trajectory. In particular, we typically want the plant to operate at a desired working point $x_S \neq 0$, and we can allow for a constant input u_S that compensates for constant disturbances.
If a desired equilibrium point (x_S, u_S) is known, it satisfies

$$x_S = Ax_S + Bu_S.$$

In this case, MPC can be used to stabilize the system around this point by introducing the shifted variables

$$\tilde{x}_k = x_k - x_S, \qquad \tilde{u}_k = u_k - u_S.$$

Substituting into the system dynamics $x_{k+1} = Ax_k + Bu_k$, we obtain

$$\tilde{x}_{k+1} = A\tilde{x}_k + B\tilde{u}_k,$$

which has exactly the same form as the original system. Therefore, in the new coordinates, the regulation problem becomes a standard stabilization problem around the origin.
The finite-horizon MPC cost can then be written as

$$V(x_0) = \min_{\tilde{\mathbf{x}},\tilde{\mathbf{u}}} \sum_{k=0}^{K-1} \left(\tilde{x}_k^\top Q\tilde{x}_k + \tilde{u}_k^\top R\tilde{u}_k\right) + \tilde{x}_K^\top S\tilde{x}_K.$$

For linear systems, this change of variables is **inconsequential**, since it does not modify the structure of the optimal control problem: the dynamics remain linear, the cost remains quadratic, and the constraints (if any) remain affine. As a result, regulating the system to (x_S, u_S) is equivalent to regulating the shifted system to the origin.

But how do we decide the steady state (x_S, u_S) to which we want to regulate? In some cases, this may be given by the problem specification, e.g., if we want to track a reference trajectory. In other cases, it can be derived from first principles, e.g., by solving the steady-state equations of the system.
In other cases, specifications indicate a desidered steady state $(x_{\text{spec}}, u_{\text{spec}})$, but the system cannot be stabilized around it. In this case, we can use MPC to find the closest stabilizable point to the desired one, by solving the following optimization problem:

$$\begin{aligned} \min_{x_S,u_S} \quad & \|x_S - x_{\text{spec}}\|_{Q_S}^2 + \|u_S - u_{\text{spec}}\|_{R_S}^2 \\ \text{subject to} \quad & \begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_S \\ u_S \end{bmatrix} = 0 \\ & (x_S, u_S) \in \mathcal{X} \times \mathcal{U}, \end{aligned}$$

where $\mathcal{X}$ and $\mathcal{U}$ are the state and input constraint sets, respectively. The condition $\begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_S \\ u_S \end{bmatrix} = 0$ ensures that the chosen point is an equilibrium of the system, while the cost function penalizes deviations from the desired specifications.
This optimization problem is typically **solved offline** and serves to compute a feasible and stabilizable steady-state target (x_S, u_S). The resulting pair is then used as the reference for the MPC regulator, which is responsible for driving the system to this point while satisfying

constraints.

Many modern control problems present **complex performance metrics** that cannot be easily captured by a quadratic cost function. For example, in chemical processes, the economic performance of the system may depend on nonlinear functions of the state and input variables, such as production rates, energy consumption, or profit margins. In such cases, we can use **economic MPC**, which allows for more general cost functions that directly reflect the economic objectives of the system.

Thus, let $\ell(x, u)$ be a general stage cost function that captures the economic performance of the system like economic losses, energy use , cost of material, etc. The economic MPC problem assumes that

- $\ell(x, u)$ is a **continuos** function;
- $\ell(x, u)$ is **lower bounded** in the domain

While remember that in MPC we were assuming that

- $g(x_S, u_S) = 0$ (i.e., the system is at equilibrium at the desired setpoint);
- $g(x, u) \geq 0$
- $g(x, u)$ is radially unbounded, continuos, zero at the origin, and strictly increasing.
- e.g. $g(x, u) = x^\top Q x + u^\top R u$.

One possible approach is to **separate the economic objective from the dynamic control problem**. In particular, the economic cost is handled at the steady-state level, while MPC is used to steer the system toward the corresponding optimal operating point.
Given a general stage cost $\ell(x, u)$, we first define the **steady-state optimization problem**

$$\begin{aligned} \min_{x_S, u_S} \quad & \ell(x_S, u_S) \\ \text{subject to} \quad & x_S = f(x_S, u_S), \\ & (x_S, u_S) \in \mathcal{X} \times \mathcal{U}. \end{aligned}$$

This problem **determines the economically optimal equilibrium of the system**.

The resulting pair (x_S, u_S) is then used as a reference for the MPC controller, which is designed to track this steady state using a standard quadratic cost. In this way, the economic objective is indirectly enforced through steady-state optimization, while the MPC ensures closed-loop stability and constraint satisfaction.

This approach is typically implemented by solving the steady-state optimization problem offline, or at a slower time scale than the MPC loop. However, this separation introduces some limitations. The closed-loop trajectory is generally **suboptimal from an economic point of view**, since the stage cost $\ell(x, u)$ is not minimized during transients. Moreover, the controller cannot exploit transient behavior to improve performance, and its response to disturbances may be economically inefficient. On the other hand, the resulting MPC problem remains a standard linear-quadratic program, which is computationally efficient and well understood.

An alternative approach is to **incorporate the economic objective** directly into the dynamic optimization problem. This leads to the formulation of **economic MPC**, in which the stage cost $\ell(x, u)$ is optimized along the entire predicted trajectory:

$$V(x) = \min_{\mathbf{x}, \mathbf{u}} \sum_{k=0}^{K-1} \ell(x_k, u_k)$$

subject to

$$x_{k+1} = f(x_k, u_k), \quad x_0 = x, \quad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}.$$

In contrast to the previous approach, there is **no separation** between steady-state optimization and tracking. The controller directly optimizes the economic performance over the prediction horizon, resulting in a *single time-scale* scheme in which both transient and steady-state behavior are determined simultaneously.

As a consequence, the controller computes control actions that generate an economically optimal trajectory, rather than simply driving the system toward a precomputed steady state. In particular, the optimal operation may not correspond to a fixed equilibrium, and the controller may intentionally exploit transient behavior if this improves the overall performance.
This increased flexibility comes at the price of a **more challenging optimization problem**. Since the cost $\ell(x, u)$ is in general nonlinear and not necessarily convex, the resulting optimal control problem may be non-convex and harder to solve in real time. Moreover, standard quadratic Lyapunov arguments used in tracking MPC no longer apply directly, and additional conditions are required to guarantee stability.

On the other hand, economic MPC is particularly well suited for nonlinear systems and applications in which performance is naturally described by non-quadratic criteria, such as energy efficiency or economic profit. In these cases, directly optimizing $\ell(x, u)$ allows the controller to fully exploit the system dynamics and constraints to achieve improved closed-loop performance.

A **key issue** with economic MPC arises from the fact that the stage cost $\ell(x, u)$ is not designed to enforce convergence to a steady state. In particular, it may happen that there exist pairs (x, u) that are not equilibria of the system, i.e.,

$$x \neq f(x, u),$$

for which

$$\ell(x, u) < \ell(x_S, u_S),$$

where (x_S, u_S) is an optimal steady state.
This situation is not pathological; on the contrary, it is quite common in practice. Since $\ell(x, u)$ reflects instantaneous economic performance, **it may be advantageous for the system to operate away from equilibrium**, for instance by exploiting transient dynamics or time-varying operating conditions.

As a consequence, convergence to a steady state is no longer guaranteed nor even desired in general. The optimal closed-loop behavior may correspond to trajectories that do not settle at an equilibrium, such as periodic or more complex operating regimes.
This highlights a **fundamental difference** with standard tracking MPC: convergence to a steady state is not part of the control specification. Instead, the objective is to optimize economic performance over time, possibly at the expense of steady-state operation.

However, this increased flexibility comes with **important challenges**. First, operating away from equilibrium can lead to behaviors that are harder to analyze and predict. In particular, stability and boundedness of the closed-loop trajectories are not automatically guaranteed. Second, without additional conditions or constraints, the controller may generate trajectories that are economically optimal in the short term but undesirable from a safety or robustness perspective.

A fundamental design choice in economic MPC is the length of the prediction horizon. In principle, the most natural formulation is the infinite-horizon problem

$$\sum_{k=0}^{\infty} \ell(x_k, u_k),$$

which directly captures the long-term economic performance of the system.
The infinite-horizon formulation has a clear economic interpretation, as it reflects the cumulative cost over an indefinite future. This is particularly meaningful in continuous operation settings, where there is no natural terminal time and performance is evaluated over the long run. Moreover, constraints are enforced over the entire trajectory, which ensures boundedness of the closed-loop system by construction.
However, the infinite-horizon problem is infinite-dimensional and therefore computationally intractable in practice. For this reason, it cannot be solved directly in real time.

As a result, practical implementations rely on a finite-horizon approximation of the form

$$\sum_{k=0}^{K-1} \ell(x_k, u_k).$$

This leads to a tractable optimization problem that can be solved online within the MPC framework.
In standard tracking MPC, a finite horizon provides a good approximation of the infinite-horizon problem, since the system is driven toward an equilibrium and the contribution of the tail beyond the horizon can be neglected or approximated. In economic MPC, this argument no longer holds. Since the optimal behavior may not converge to a steady state, the portion of the trajectory beyond the horizon can have a significant impact on performance.

This mismatch introduces several potential issues. First, the controller may favor trajectories that yield low cost within the prediction horizon but lead to poor behavior afterward, as future consequences are not accounted for. In particular, the system may move toward regions from which recovery is difficult or expensive, resulting in undesirable or even unstable behavior.
Second, feasibility problems may arise at the end of the horizon. Due to state and input constraints, not all states reached within the horizon can be extended to admissible infinite trajectories. This can lead to situations in which the optimization problem becomes infeasible at future time steps.

A common approach to mitigate these issues is to augment the finite-horizon problem with a terminal constraint. In particular, one enforces that the predicted trajectory reaches a feasible steady state at the end of the horizon, i.e.,

$$x_K = x_S, \qquad u_K = u_S,$$

where (x_S, u_S) is an admissible equilibrium of the system.
This modification preserves the computational tractability of the finite-horizon formulation, while introducing a mechanism to guarantee that the predicted trajectory can be extended beyond the horizon in a feasible way. In other words, by constraining the terminal state to an equilibrium, one ensures that there exists a feasible infinite-horizon continuation, thereby recovering *recursive feasibility*.

From this perspective, the terminal constraint provides an explicit characterization of what constitutes a feasible infinite-horizon trajectory: a trajectory is admissible if it can be steered to a steady state that satisfies the system dynamics and constraints.

However, the introduction of a terminal constraint also raises important questions regarding the resulting closed-loop behavior. In particular, it is no longer immediate whether the closed-loop system is stable, or whether the resulting trajectory achieves optimal economic performance. The terminal constraint enforces convergence to a steady state, which may restrict the ability of the controller to exploit economically favorable transient or non-equilibrium operation.

Finally, it is important to emphasize that, as in any MPC scheme, the closed-loop system does not follow the predicted trajectory exactly. At each time step, only the first control input is applied,

$$u_k = u^*(x_k),$$

and the optimization problem is solved again at the next step with updated state information. As a result, the actual closed-loop trajectory is generated by the repeated solution of the finite-horizon problem, and not by the open-loop prediction computed at a single time instant.

To better illustrate the difference between standard tracking MPC and economic MPC, consider the following linear system

$$x_{k+1} = Ax_k + Bu_k, \qquad A = \begin{bmatrix} \frac{1}{2} & 1 \\ 0 & \frac{3}{4} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

with input constraint

$$u \in \mathcal{U} = [-1, 1],$$

and economic stage cost

$$\ell(x, u) = q^\top x + ru, \qquad q = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, \quad r = -10.$$

The first step consists in computing the optimal steady state by solving

$$\begin{aligned} \min_{x_S, u_S} \quad & \ell(x_S, u_S) \\ \text{subject to} \quad & x_S = Ax_S + Bu_S, \\ & u_S \in \mathcal{U}. \end{aligned}$$

To find the steady states we need to solve

$$(I - A)x_S = Bu_S,$$

which gives

$$\begin{bmatrix} \frac{1}{2} & -1 \\ 0 & \frac{1}{4} \end{bmatrix} \begin{bmatrix} x_{S,1} \\ x_{S,2} \end{bmatrix} = \begin{bmatrix} 0 \\ u_S \end{bmatrix}$$

Since the cost is linear and the constraints define a polyhedral set, this is a linear program, and the optimum is attained at an extreme point of the feasible set. Thus we only need to evaluate the cost at the two vertices of the input constraint, which gives

$$\text{if } u_S = -1, \quad x_S = \begin{bmatrix} -4 \\ -8 \end{bmatrix}, \quad \ell(x_S, u_S) = 18,$$

$$\text{if } u_S = 1, \quad x_S = \begin{bmatrix} 4 \\ 8 \end{bmatrix}, \quad \ell(x_S, u_S) = -18.$$

The solution (x_S^*, u_S^*) represents the economically optimal equilibrium.
Once (x_S^*, u_S^*) is determined, two fundamentally different control strategies can be constructed.

In the standard MPC tracking approach, the optimal steady state is used as a reference. Defining the shifted variables

$$\tilde{x}_k = x_k - x_S^*, \qquad \tilde{u}_k = u_k - u_S^*,$$

the controller minimizes a quadratic tracking cost of the form

$$g(\tilde{x}, \tilde{u}) = \tilde{x}^\top Q \tilde{x} + \tilde{u}^\top R \tilde{u},$$

driving the system to (x_S^*, u_S^*) as fast as possible. A terminal constraint $\tilde{x}_K = 0$ (equivalently $x_K = x_S^*$) is typically imposed to ensure stability. The resulting optimization problem is a quadratic program.

In contrast, economic MPC directly uses the economic cost in the receding horizon problem:

$$\min_{\mathbf{x},\mathbf{u}} \sum_{k=0}^{K-1} \ell(x_k, u_k)$$

subject to

$$x_{k+1} = Ax_k + Bu_k, \qquad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U},$$

together with a terminal constraint $x_K = x_S^*$ to guarantee feasibility. In this case, since both the dynamics and the cost are linear, the resulting problem is a linear program.

The key difference between the two approaches lies in the objective being optimized. Tracking MPC enforces convergence to the economically optimal steady state and prioritizes stability and regulation performance, but does not optimize the economic cost during transients. Economic MPC, on the other hand, optimizes the economic objective along the entire trajectory, potentially exploiting transient behavior to improve performance.

From a computational perspective, tracking MPC leads to a quadratic program, which is typically well-conditioned and easier to solve reliably in real time. Economic MPC may lead to linear or nonlinear programs depending on the form of $\ell(x, u)$, and can be more challenging, although in this particular example it reduces to a linear program.

Now we want to analyze the closed loop behavior. In particular we want to understand if the closed loop stability is guaranteed and if the economic cost is minimez. In standard tracking MPC, the terminal constraint plays a crucial role in proving asymptotic stability. Indeed, when the cost is built from a positive definite tracking stage cost

$$g(\tilde{x}, \tilde{u}),$$

with

$$g(0, 0) = 0, \qquad g(\tilde{x}, \tilde{u}) > 0 \quad \text{for } \tilde{x} \neq 0,$$

the finite-horizon optimal value function can be used as a Lyapunov function for the closed-loop system. The key argument is the same shifted-sequence argument already discussed in the previous chapter. If the optimal predicted trajectory satisfies the terminal constraint

$$\tilde{x}_K = 0,$$

then, after applying the first optimal input, a feasible candidate sequence at the next time step is obtained by shifting the previously optimal sequence and appending the equilibrium input $\tilde{u} = 0$. Since the appended terminal stage cost is zero, one obtains

$$W(f(\tilde{x}, u_0^*(\tilde{x}))) \leq W(\tilde{x}) - g(\tilde{x}, u_0^*(\tilde{x})) \leq W(\tilde{x}),$$

which shows that W decreases along the closed-loop trajectories and is therefore a valid Lyapunov function.

The natural question is whether the same idea can be applied to economic MPC. Suppose that we consider the finite-horizon economic MPC problem with terminal constraint

$$x_K = x_S,$$

where (x_S, u_S) is an admissible steady state. The same shifted-sequence construction remains feasible: after applying the first optimal control input, one can shift the optimal predicted trajectory and append the steady-state input u_S at the end of the horizon. Therefore, feasibility is preserved. However, the cost argument changes in a fundamental way.
Indeed, for economic MPC the stage cost is $\ell(x, u)$, which is not positive definite with respect to the steady state and is not, in general, minimized at (x_S, u_S). As a consequence, the shifted-sequence argument only yields

$$W(f(x, u_0^*(x))) \leq W(x) - \ell(x, u_0^*(x)) + \ell(x_S, u_S).$$

This inequality is not sufficient to conclude that W decreases. In fact, it may happen that

$$\ell(x_S, u_S) > \ell(x, u_0^*(x)),$$

so the right-hand side can be larger than $W(x)$. Therefore, unlike in tracking MPC, the optimal finite-horizon value function is *not* automatically a Lyapunov function for the closed-loop system.

This observation has an important conceptual consequence. In economic MPC, asymptotic stability of the steady state is not guaranteed by the terminal constraint alone. The terminal constraint guarantees recursive feasibility, but not necessarily convergence to the equilibrium. This is perfectly consistent with the economic interpretation of the problem: the controller is asked to optimize economic performance, not to minimize the distance from the steady state. If operating away from equilibrium is economically more favorable, then the closed-loop system may prefer persistent transient behavior or even non-equilibrium operating regimes.

For this reason, asymptotic convergence to the steady state is not always the most relevant objective in economic MPC. In some applications, one may actually achieve a lower long-run cost by not converging to an equilibrium. For nonlinear systems, it is even possible that economically preferable periodic trajectories exist, so that oscillatory operation can outperform steady-state operation. Hence, from the viewpoint of economic MPC, the relevant performance question is often not whether the state converges to x_S, but whether the closed-loop system achieves good *average* economic performance over time.

This leads to a weaker, but very meaningful, notion of closed-loop performance. Consider the closed-loop trajectory generated by economic MPC with terminal constraint x_S. Under suitable assumptions, one can prove the following asymptotic average performance bound:

$$\limsup_{t\to\infty} \frac{1}{t} \sum_{k=0}^{t-1} \ell(x_k, u_k) \leq \ell(x_S, u_S).$$

This result states that the asymptotic average economic performance of the closed loop is no worse than the cost of operating permanently at the chosen steady state. In other words, even if the trajectory does not converge to (x_S, u_S), its long-run average performance is at least as good as the steady-state economic benchmark.

The proof again relies on the terminal constraint and on the shifted-sequence argument. From feasibility of the shifted trajectory, one obtains

$$W(f(x, u_0^*(x))) \leq W(x) - \ell(x, u_0^*(x)) + \ell(x_S, u_S).$$

Summing this inequality along the closed-loop trajectory for $k = 0, \ldots, t-1$ yields

$$\sum_{k=0}^{t-1} \ell(x_k, u_k) \leq t\, \ell(x_S, u_S) + W(x_0) - W(x_t).$$

Dividing by t gives

$$\frac{1}{t} \sum_{k=0}^{t-1} \ell(x_k, u_k) \leq \ell(x_S, u_S) + \frac{W(x_0) - W(x_t)}{t}.$$

If the value function remains bounded along the closed loop, the last term vanishes as $t \to \infty$, from which the average performance bound follows.

It is important to stress the meaning of this result. The function W is not a Lyapunov function here, because it is not required to decrease at every time step. Instead, it acts as a storage-like quantity that allows us to compare the accumulated closed-loop cost with the steady-state benchmark. Thus, the conclusion is not asymptotic stability, but an *asymptotic average cost guarantee*.

We can now summarize the main message of this discussion. In economic MPC, a terminal constraint is essential to guarantee recursive feasibility of the receding-horizon scheme. However, unlike in tracking MPC, the terminal constraint alone is not enough to guarantee asymptotic stability of the optimal steady state. The closed-loop system may generate economically efficient trajectories that do not converge to an equilibrium, and this is not necessarily undesirable. In fact, even when the trajectory eventually converges to the steady state, economic MPC may still outperform standard tracking MPC because it optimizes the transient behavior as well. Over long operating periods, and especially in the presence of recurrent disturbances, this transient economic advantage can accumulate and lead to significantly improved overall performance.

Additional assumptions can be introduced to recover asymptotic stability of the steady state in economic MPC. These conditions are stronger than those used in standard tracking MPC and are not automatic consequences of the basic formulation. Their role is precisely to connect economic optimality with a suitable notion of dissipativity or storage of the system, so that the closed-loop behavior can again be related to equilibrium convergence. In the absence of such additional assumptions, the most natural guarantee remains the asymptotic average performance property discussed above.

4 Feedback optimization

4.1 Optimal steady-state regulation

In many automation problems, the control objective is not only to stabilize the plant, but to drive it toward an *optimal operating point*. This operating point is typically characterized by steady-state specifications rather than by transient performance requirements.

The **setting** we have in mind is the following. The plant is nonlinear, but it is either already stable or has been pre-stabilized by a lower-level controller. Therefore, the main control task is no longer to stabilize an unstable system from scratch, but to **determine and maintain the best admissible steady state**. This is particularly relevant when the plant must operate continuously around an equilibrium and performance is mainly determined by the chosen operating point.

A second important feature is that, in many applications, an accurate dynamic model is not available. Even when a rough dynamical description exists, it may be too uncertain to support a reliable model-based dynamic optimization over a prediction horizon. On the other hand, steady-state relations are often easier to obtain from physical insight, experiments, maps, or data. For this reason, one may prefer to formulate the control objective directly in terms of steady-state optimization.
The steady-state specifications are usually expressed in terms of optimality criteria. Typical examples are:

- maximizing yield;
- minimizing economic cost;
- reducing CO_2 emissions.

Hence, instead of assigning a set-point a priori, one wants to *compute* the best steady state according to a given performance index.

This optimization is rarely unconstrained. In practice, several manipulated inputs must be selected simultaneously, and each of them is subject to physical or operational limits. Moreover, the desired operating point must satisfy several steady-state constraints, such as safety limits, balance equations, admissibility conditions, or exact power and mass balances. Therefore, the target selection problem is naturally a **constrained steady-state optimization problem**.

Another crucial aspect is that the optimal steady state is generally not fixed once and for all. It depends on exogenous factors that affect the plant equilibrium. These may include external disturbances, slowly varying demand, ambient conditions, or unknown plant parameters. As these quantities change, the economically or operationally optimal steady state changes as well. The controller must then adapt the plant operation so as to regulate the system around the new optimal equilibrium.

Two representative application domains help clarify the nature of the problem.
In *grid frequency control*, one must decide how to activate available reserves, that is, how much generation should ramp up or down. The external demand is time-varying and acts as an exogenous signal. At the same time, one seeks low operating cost while satisfying the exact steady-state power balance. Thus, the main question is how to choose the steady-state allocation of generation in an optimal and constraint-satisfying way.
In *process systems*, for instance in compressor networks, the mapping from the manipulated variables to the steady-state operating condition can be highly nonlinear and difficult to model dynamically. There may be several tunable parameters and set-points, together with efficiency

objectives, safety constraints, and actuator limitations. Again, the core task is to find and maintain the best admissible steady state.

Difference from Economic MPC. At first sight, this viewpoint may seem close to Economic MPC, since both approaches use economic or performance-oriented criteria rather than purely quadratic tracking objectives. The difference is conceptual and important.
In Economic MPC, the economic stage cost is optimized *along the whole predicted trajectory*. Therefore, transient behavior is part of the optimization problem: the controller may intentionally exploit transients if this improves the overall economic performance. For this reason, Economic MPC requires a dynamic model and solves a finite-horizon dynamic optimization problem online. In optimal steady-state regulation, instead, the main objective is to optimize the *equilibrium* itself. The transient is not the primary object of optimization. One assumes that the plant is stable or pre-stabilized, and one uses feedback mainly to steer the system toward the optimal admissible steady state and to keep it there despite changes in disturbances or parameters. In this sense, feedback optimization is closer to a real-time steady-state target adaptation layer than to full dynamic economic optimization.

Therefore, the two approaches differ in what is being optimized:

- **Economic MPC:** optimize the economic performance of the entire transient trajectory;
- **Optimal steady-state regulation / feedback optimization:** optimize the steady-state operating point and regulate the plant around it.

This distinction is especially relevant when the dynamic model is inaccurate but the steady-state behavior is sufficiently known. In that case, feedback optimization remains meaningful, while Economic MPC may be difficult to formulate reliably.

4.2 Fast-stable plant

We now introduce the basic setting for feedback optimization in the case of a *fast and stable* plant. The idea is that the plant dynamics are sufficiently fast, and already stable, so that from the viewpoint of the upper optimization layer the dominant object is the steady-state input–output relation.

We assume a continuous-time, linear time-invariant plant of the form

$$\dot{x} = Ax + Bu + Ew, \qquad y = Cx + Du,$$

where x is the state, u is the manipulated input, w is an exogenous variable or disturbance, and y is the controlled output or performance variable of interest.

The key assumption is that the plant is *exponentially stable*. This means that, for constant inputs u and constant disturbances w, the state converges exponentially fast to a unique equilibrium. Therefore, **after transients have died out, the output is determined only by the constant values of u and w**. This motivates the introduction of the *steady-state map*

$$y = Hu + Lw.$$

In the general nonlinear case, the same idea is expressed by a steady-state relation of the form

$$y = h(u; w),$$

where h is a possibly nonlinear map that associates to each constant input–disturbance pair the corresponding steady-state output.

Let us now derive the matrices H and L in the linear case. At steady state we must have $\dot{x} = 0$, hence

$$0 = Ax + Bu + Ew.$$

If A is invertible, we can solve for the equilibrium state:

$$x = -A^{-1}Bu - A^{-1}Ew.$$

Substituting this expression into the output equation gives

$$y = Cx + Du = C\left(-A^{-1}Bu - A^{-1}Ew\right) + Du.$$

Rearranging the terms yields

$$y = \left(-CA^{-1}B + D\right)u - CA^{-1}Ew.$$

Therefore, the steady-state gains are

$$H = -CA^{-1}B + D, \qquad L = -CA^{-1}E.$$

So H is the steady-state gain from the manipulated input u to the output y, while L is the steady-state gain from the exogenous signal w to the output y.

This also answers the question about the conditions on A. In order to define the steady-state map in this form, A must be invertible. In particular, if the plant is exponentially stable, then all eigenvalues of A have strictly negative real part. Hence 0 cannot be an eigenvalue of A, and therefore A is automatically nonsingular. So invertibility of A is not an additional restriction here: it already follows from exponential stability.

The conclusion is that, f**for a fast and exponentially stable plant**, the dynamic relation between u and y can be replaced at steady state by the static map

$$y = Hu + Lw,$$

which is the object that will be used in the feedback optimization layer.

4.3 "Certainty-equivalence" design

Given the steady-state map

$$y = h(u; w),$$

a natural first idea is to formulate the steady-state requirements directly as a static optimization problem. Let $\Phi(u, y)$ denote the performance index to be minimized, let $\mathcal{U}$ be the set of admissible inputs, and let $\mathcal{Y}$ be the set of admissible steady-state outputs. The steady-state specifications can then be written as

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

This formulation expresses exactly the control objective discussed above: choose the input u so that the plant settles at a feasible steady state y, while optimizing the desired economic or operational criterion.

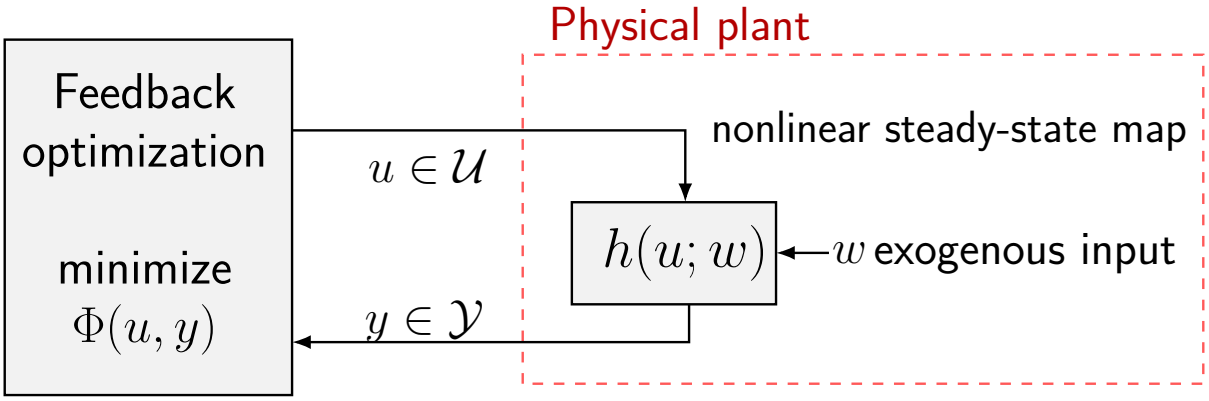

Figure 6: Certainty-equivalence view of steady-state optimization. The optimizer computes the input $u \in \mathcal{U}$ using the steady-state relation $y = h(u; w)$ so as to minimize the performance index $\Phi(u, y)$ while satisfying the steady-state constraints $y \in \mathcal{Y}$.

If the exogenous quantity w were known and the map h were exactly available, then one could eliminate the variable y and solve the reduced problem

$$u^\star = \arg\min_{u \in \mathcal{U}} \tilde{\Phi}(u) \qquad \text{with} \qquad \tilde{\Phi}(u) := \Phi\big(u, h(u; w)\big),$$

subject to

$$h(u; w) \in \mathcal{Y}.$$

Once the optimizer $u^\star$ has been computed, one simply applies it to the plant. This is the standard *certainty-equivalence* or *feedforward* approach: one behaves as if the uncertain quantities were known with certainty and as if the plant steady-state model were exact.

Conceptually, this is appealing because it reduces the problem to a classical nonlinear optimization task. However, this approach has two important weaknesses.
First, it requires accurate knowledge of the exogenous input w. In practice, w may represent an unknown disturbance, a slowly varying demand, or an uncertain operating condition. If the value used in the optimization differs from the true one, then the computed input $u^\star$ is generally not the true optimizer for the physical plant. As a consequence, the achieved steady state may be suboptimal or may even violate the output constraints.

Second, the approach requires an accurate steady-state model h. If the map used in the optimization does not match the actual plant steady-state relation, then the predicted output

$$y = h(u; w)$$

is not the one that will actually be produced by the plant. Hence, even when the optimization problem is solved exactly, the resulting input may fail to achieve the desired steady-state specifications on the real system.

So the usual feedforward problem is clear: solving the static optimization problem is not enough when the disturbance is uncertain and the model is imperfect. The optimizer is optimal only for the *model-based problem*, not necessarily for the physical plant. This is precisely the motivation for introducing feedback into the optimization loop.

A natural question then arises: *how can we design feedback controllers that yield the desired autonomous closed-loop behavior?* The idea developed in feedback optimization is to combine two ingredients:

- an algebraic input/output steady-state map

$$y = h(u; w),$$

  valid when w is constant or slowly changing;

- classical iterative nonlinear optimization algorithms.

Rather than solving the steady-state optimization problem once and applying the resulting input in open loop, we construct a controller that updates the input iteratively on the basis of measured plant outputs. In this way, the optimization algorithm and the physical plant become interconnected and the optimizer is embedded into the feedback loop. The resulting closed-loop system should drive the plant toward a steady state that satisfies the constraints and is optimal for the *actual* plant, not only for the nominal model.

4.4 Interconnecting plant and algorithms

This viewpoint can be understood by comparing two different implementations.

In the first implementation, one keeps the optimization algorithm and the plant separate. The algorithm interacts only with a model of the steady-state map and solves

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

The model returns the optimizer $u^\star$, and this value is then applied to the plant. This is exactly the feedforward architecture discussed above. Its weakness is that the optimization loop closes around the *model*, while the real plant appears only at the end, as a system to which the precomputed command is sent. Therefore, any mismatch between model and plant directly affects the achieved performance.

The alternative is to interconnect the optimization algorithm with the physical plant itself. In this case, the algorithm produces an input u, the plant responds with an output y, and this measured output is fed back to the algorithm. The optimization method is no longer driven only by a nominal model prediction, but by the actual plant behavior. As a consequence, the closed-loop system formed by

$$\text{optimization algorithm} + \text{plant}$$

becomes the main object of analysis and design.

This is the key conceptual step of feedback optimization. The optimization algorithm is no longer viewed as an offline numerical routine that computes a set-point. Instead, it is implemented dynamically and interconnected with the plant in feedback. The plant acts as part of the optimization loop, and the measured output provides the information needed to compensate unknown disturbances and model mismatch.

In summary, the transition is the following:

- in feedforward optimization, the algorithm solves the steady-state problem on a model and sends the resulting optimizer to the plant;
- in feedback optimization, the algorithm is directly interconnected with the plant, and the plant measurements are used online to steer the iterative process toward the desired optimum.

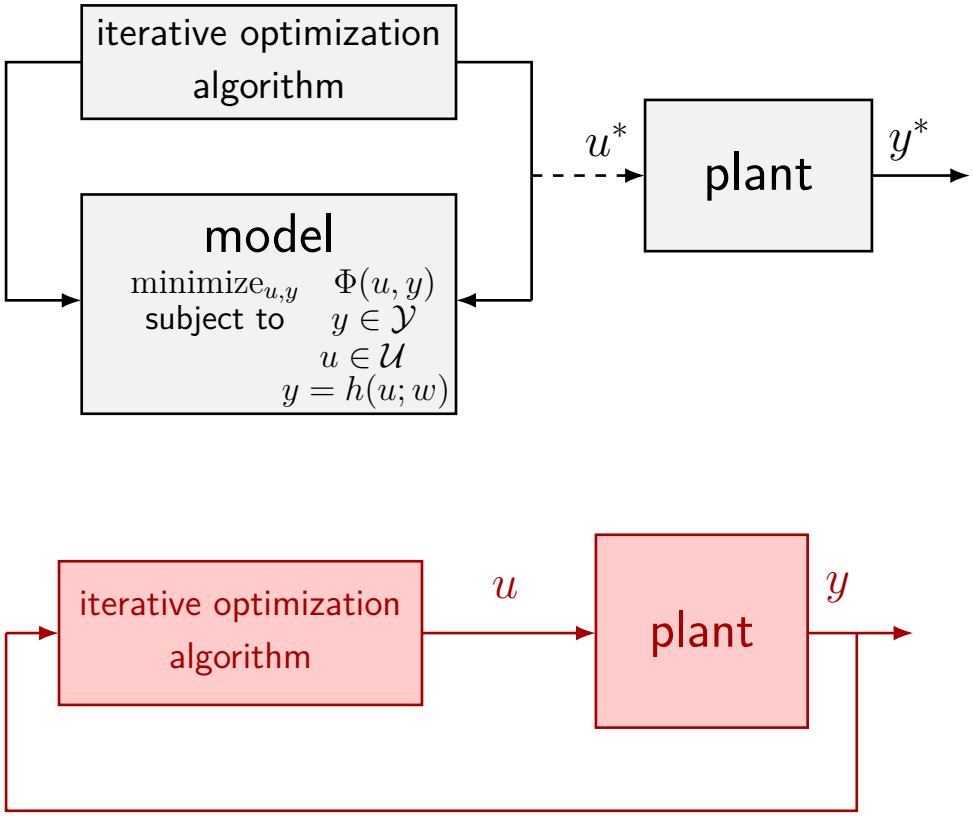

Figure 7: Comparison between model-based feedforward optimization and feedback optimization. In the upper scheme, the iterative optimization algorithm is closed around the model and produces the command $u^\star$ that is then sent to the plant. In the lower scheme, the optimization algorithm is directly interconnected with the physical plant through the measured output y, thereby forming an autonomous feedback loop.

This interconnection is what allows one to move beyond certainty-equivalence design and obtain an autonomous closed-loop behavior with improved robustness to uncertainty in w and in the steady-state map h.

4.5 Online feedback optimization

The previous discussion motivates a shift from static feedforward optimization to *online feedback optimization*. The idea is to implement an optimization algorithm as a dynamical system interacting directly with the plant, so that the plant measurements continuously correct the optimization process. In this way, the input is not computed once and for all from an uncertain model, but is updated online on the basis of the actual closed-loop behavior.

The steady-state optimization problem of interest is

$$\begin{aligned} \min_{u,y} \quad & \Phi(u,y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u;w). \end{aligned}$$

Using the steady-state relation, this problem can be equivalently written only in terms of the

decision variable u as

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \Phi\big(u, h(u; w)\big) \\ \text{s.t.} \quad & h(u; w) \in \mathcal{Y}. \end{aligned}$$

So the plant enters the optimization problem through the algebraic steady-state map $y = h(u; w)$, while the constraints may act both directly on the input and indirectly on the output.

Once the problem has been written in this form, many classical iterative methods become available. In particular, the most relevant class for feedback optimization is that of *first-order optimization algorithms*, that is, methods based on gradient information or generalized descent directions. Typical examples are:

- gradient descent combined with penalty or barrier terms;
- projected gradient methods;
- primal–dual dynamics or primal–dual flows.

The reason these methods are attractive is that they admit iterative implementations that can naturally be interconnected with the plant. Instead of solving the optimization problem to completion at each step, one performs repeated correction steps, and the plant output provides the information needed to drive the iterations toward the desired optimum.

4.6 First-order optimization algorithms

Let us now look more closely at the class of first-order methods. Consider again the constrained steady-state problem

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

Equivalently, after eliminating y through the steady-state map, we obtain

$$\begin{aligned} \min_{u \in \mathcal{U}} \quad & \Phi\big(u, h(u; w)\big) \\ \text{s.t.} \quad & h(u; w) \in \mathcal{Y}. \end{aligned}$$

This equivalent formulation is useful because it shows that the optimization variable can be taken to be only u, while the output constraint is enforced through the plant steady-state relation.

At this point, the problem has the structure of a nonlinear constrained optimization problem, and one may attack it with standard first-order techniques. The main issue is how to handle the constraints. A first possibility is to absorb the output constraint into the cost by adding a suitable extra term. This leads to gradient methods with *penalty* or *barrier* functions.

4.7 Gradient descent with penalty / barrier

Consider the constrained problem

$$\begin{aligned} \min_{u,y} \quad & \Phi(u, y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u; w). \end{aligned}$$

We focus here on the output constraint $y \in \mathcal{Y}$. A standard way to handle it is to transform the constrained problem into an unconstrained or less constrained one by modifying the cost function. Two classical constructions are the penalty approach and the barrier approach.

Penalty function. In the penalty approach, one introduces an additional term $p(y)$ in the objective and solves

$$\min_{u,y} \ \Phi(u, y) + p(y).$$

The function $p(y)$ is chosen so that it is small or zero when the constraint is satisfied, and increases when the output violates the admissible set $\mathcal{Y}$. In this way, constraint violations are discouraged, but not made impossible. Small violations may still occur if they lead to a lower total cost.

This approach is often attractive because it leads to a smooth unconstrained optimization problem, which is easy to handle with gradient-based methods. However, the price to pay is that feasibility is not enforced exactly: one only penalizes infeasibility. Therefore, the obtained solution may slightly violate the constraint, depending on the shape and weight of the penalty.

(a) Penalty function. (b) Barrier function.

Figure 8: Qualitative comparison between penalty and barrier functions used to enforce the output constraint $y \in \mathcal{Y}$. A penalty function remains finite outside the feasible set and discourages violations without excluding them completely. A barrier function instead becomes unbounded at the boundary of the feasible set and is undefined outside it, thereby forcing the iterates to remain strictly feasible.

Barrier function. In the barrier approach, one instead introduces a function $b(y)$ and solves

$$\min_{u,y} \ \Phi(u, y) + b(y).$$

Now the function $b(y)$ is defined only inside the feasible set and tends to $+\infty$ as y approaches the boundary of $\mathcal{Y}$. As a consequence, the optimizer is forced to remain in the interior of the feasible region.

This gives a more conservative behavior than the penalty approach. Since the barrier becomes very large near the boundary, the algorithm tends to stay away from it. Moreover, the formulation is not even defined outside the feasible set, so one must initialize and operate the algorithm inside the admissible region. The benefit is that feasibility is preserved by construction, at least in the ideal continuous-time or exact-iteration picture.

Comparison. The two approaches reflect two different ways of dealing with constraints:

- with a *penalty function*, small violations are acceptable, since infeasibility is merely discouraged;

- with a *barrier function*, one is conservative and never reaches the boundary, because leaving the feasible region is forbidden by the formulation itself.

These ideas are particularly useful in feedback optimization because they turn constrained optimization into a form that can be handled by simple first-order iterative laws. The optimization algorithm then updates the input u by descending along the modified objective, while the plant provides the output y entering the cost and the constraint-handling terms.

4.8 Unconstrained gradient descent

Before applying gradient methods to the feedback optimization problem, it is useful to recall the basic convergence property of gradient descent in the unconstrained case. Consider the problem

$$\min_x \ \phi(x),$$

and assume that ϕ is twice continuously differentiable and has bounded second derivative, namely

$$\nabla^2\phi(x) \preceq MI \qquad \forall x,$$

for some constant $M > 0$. This means that the gradient of ϕ is Lipschitz continuous with Lipschitz constant bounded by M.
The gradient descent iteration is

$$x_{k+1} = x_k - \alpha\nabla\phi(x_k),$$

where $\alpha > 0$ is the stepsize. Under the assumption above, this iteration converges to a local minimum of ϕ provided that

$$\alpha < \frac{2}{M}.$$

The reason is seen by expanding ϕ around the current iterate x_k. By Taylor's formula,

$$\phi(x_{k+1}) = \phi(x_k) + \nabla\phi(x_k)^\top(x_{k+1} - x_k) + \frac{1}{2}(x_{k+1} - x_k)^\top\nabla^2\phi(z)(x_{k+1} - x_k),$$

where z is a point on the segment joining x_k and x_{k+1}. Substituting the gradient descent update

$$x_{k+1} - x_k = -\alpha\nabla\phi(x_k),$$

we obtain

$$\phi(x_{k+1}) = \phi(x_k) - \alpha\|\nabla\phi(x_k)\|^2 + \frac{\alpha^2}{2}\nabla\phi(x_k)^\top\nabla^2\phi(z)\nabla\phi(x_k).$$

Using the bound $\nabla^2\phi(z) \preceq MI$, the last term can be bounded as

$$\nabla\phi(x_k)^\top\nabla^2\phi(z)\nabla\phi(x_k) \le M\|\nabla\phi(x_k)\|^2,$$

and therefore

$$\phi(x_{k+1}) \le \phi(x_k) - \alpha\|\nabla\phi(x_k)\|^2 + \frac{\alpha^2 M}{2}\|\nabla\phi(x_k)\|^2.$$

Equivalently,

$$\phi(x_{k+1}) \le \phi(x_k) - \alpha\left(1 - \frac{\alpha M}{2}\right)\|\nabla\phi(x_k)\|^2.$$

Hence, if

$$\alpha < \frac{2}{M},$$

the coefficient in parentheses is positive, and so

$$\phi(x_{k+1}) < \phi(x_k) \qquad \text{whenever } \nabla\phi(x_k) \neq 0.$$

So the sequence $\phi(x_k)$ is decreasing along the iterations until the gradient vanishes. This is the key fact behind convergence of gradient descent: the cost decreases monotonically, and the iterates are driven toward the set of stationary points

$$\{x : \nabla\phi(x) = 0\}.$$

Under suitable additional assumptions, this implies convergence to a local minimum.

4.9 Gradient descent with penalty

We now apply this idea to the steady-state optimization problem in the presence of output constraints handled through a penalty term. Starting from

$$\begin{aligned} \min_{u,y} \quad & \Phi(u,y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u;w), \end{aligned}$$

we incorporate the constraint $y \in \mathcal{Y}$ into the objective through a penalty function $p(y)$ and obtain the problem

$$\min_{u\in\mathcal{U}} \ \Phi\big(u, h(u;w)\big) + p\big(h(u;w)\big).$$

Here the admissible input set $\mathcal{U}$ is kept explicitly, while the output constraint is treated by penalization.
To derive a gradient algorithm, define the penalized objective

$$J(u) := \Phi\big(u, h(u;w)\big) + p\big(h(u;w)\big).$$

Then the gradient descent iteration is

$$u_{k+1} = u_k - \alpha\nabla J(u_k).$$

Using the chain rule, the gradient of J is

$$\nabla J(u) = \nabla_u\Phi(u, h(u;w)) + \nabla_u h(u;w)^\top \nabla_y\Phi(u, h(u;w)) + \nabla_u h(u;w)^\top \nabla_y p(h(u;w)).$$

Therefore, the gradient descent algorithm becomes

$$u_{k+1} = u_k - \alpha\Big(\nabla_u\Phi(u_k, h(u_k;w)) + \nabla_u h(u_k;w)^\top \nabla_y\Phi(u_k, h(u_k;w)) + \nabla_u h(u_k;w)^\top \nabla_y p(h(u_k;w))\Big).$$

This is the ideal algorithm one would write if the steady-state map h were exactly known. However, in feedback optimization we do not want to evaluate the cost only on the basis of the model output $h(u_k;w)$; rather, we want to use the *measured* plant output. Denoting by y_k the measured output corresponding to the current input u_k, the update law becomes

$$u_{k+1} = u_k - \alpha\Big(\nabla_u\Phi(u_k, y_k) + \nabla_u h(u_k;w)^\top \nabla_y\Phi(u_k, y_k) + \nabla_u h(u_k;w)^\top \nabla_y p(y_k)\Big).$$

This is the *gradient descent controller*: the optimization step is corrected using the actual output measurement y_k, while the Jacobian $\nabla_u h(u_k;w)$ is still taken from the model. The input is finally constrained to lie in $\mathcal{U}$, for instance by saturation or projection.
The structure of the controller is worth interpreting carefully:

- $\nabla_u\Phi(u_k, y_k)$ measures the direct effect of the input on the cost;
- $\nabla_y\Phi(u_k, y_k)$ measures how the cost changes with the output;

- $\nabla_y p(y_k)$ measures how strongly the penalty reacts to output constraint violations;
- $\nabla_u h(u_k; w)$ maps output sensitivities back to input directions.

So the matrix

$$\nabla_u h(u; w)^\top$$

plays a crucial role: it tells us how a change in input affects the steady-state output, and therefore how to translate output-based optimization information into an input update.

Steady-state sensitivities. The quantity

$$\nabla_u h(u; w)^\top$$

is the transpose of the Jacobian of the steady-state map with respect to the input. It is the local steady-state input–output sensitivity matrix. If the output dimension is p and the input dimension is m, then

$$\nabla_u h(u; w) \in \mathbb{R}^{p \times m},$$

and its (i, j)-th entry is

$$\frac{\partial h_i(u; w)}{\partial u_j}.$$

This derivative tells us how the i-th steady-state output changes when the j-th input is perturbed. For a linear fast-stable plant, where

$$y = Hu + Lw,$$

the steady-state sensitivity is simply constant:

$$\nabla_u h(u; w) = H.$$

So in the linear case the controller uses the steady-state gain matrix $H^\top$.

In nonlinear plants, instead, the sensitivity depends on the current operating point. Some intuitive examples are the following:

- in a compressor or process unit, the derivative describes how the steady-state flow, pressure, or efficiency changes when a valve opening or actuator set-point is modified;
- in power systems, it represents how steady-state frequency, power flow, or voltage-related quantities react to a change in generation dispatch;
- in thermal systems, it measures how the equilibrium temperature changes when heater power or coolant flow is varied.

Thus, the Jacobian $\nabla_u h(u; w)$ is exactly the information needed to connect an optimization algorithm to the physical plant at steady state.

The key point is that the controller does not require a full dynamic model of the plant. It only needs a model, estimate, or approximation of the steady-state sensitivities. This is much weaker than full model-based control and is one of the main reasons why feedback optimization is attractive in applications where the transient dynamics are uncertain but the steady-state behavior is sufficiently structured.

4.10 Projected gradient descent

Penalty and barrier methods handle the output constraint indirectly by modifying the objective function. A different possibility is to enforce the constraints explicitly through projection. Starting from

$$\begin{aligned} \min_{u,y} \quad & \Phi(u,y) \\ \text{s.t.} \quad & y \in \mathcal{Y}, \\ & u \in \mathcal{U}, \\ & y = h(u;w), \end{aligned}$$

and eliminating the variable y through the steady-state relation, we obtain

$$\min_{u} \ \Phi\big(u, h(u;w)\big)$$

subject to

$$u \in \mathcal{U}, \qquad h(u;w) \in \mathcal{Y}.$$

It is convenient to collect both feasibility requirements into a single admissible set for the input:

$$\tilde{\mathcal{U}} := \mathcal{U} \cap h^{-1}(\mathcal{Y};w),$$

where

$$h^{-1}(\mathcal{Y};w) := \{u : \ h(u;w) \in \mathcal{Y}\}.$$

Then the optimization problem can be rewritten as

$$\min_{u \in \tilde{\mathcal{U}}} \ \Phi\big(u, h(u;w)\big).$$

This suggests the use of projected gradient descent: one first takes a descent step for the cost, and then projects the result back onto the feasible set $\tilde{\mathcal{U}}$. Formally, if

$$J(u) := \Phi\big(u, h(u;w)\big),$$

the projected gradient iteration is

$$u_{k+1} = \Pi_{\tilde{\mathcal{U}}}\Big(u_k - \alpha \nabla J(u_k)\Big),$$

where $\Pi_{\tilde{\mathcal{U}}}$ denotes the Euclidean projection onto $\tilde{\mathcal{U}}$.
The appealing feature of this method is that, unlike a penalty formulation, it tries to maintain feasibility explicitly. The difficulty, however, is hidden in the projection step. While projection onto the input set $\mathcal{U}$ may be easy, the set

$$h^{-1}(\mathcal{Y};w)$$

is generally complicated. Even when $\mathcal{Y}$ is a simple set, its inverse image through a nonlinear steady-state map may be highly nonconvex and may have a complicated geometry. As a consequence, projecting onto

$$\tilde{\mathcal{U}} = \mathcal{U} \cap h^{-1}(\mathcal{Y};w)$$

can be numerically difficult and conceptually unpleasant. This is exactly the issue highlighted by the figures: a simple admissible set in the output space may correspond, in the input space, to a distorted and difficult set.

To make this approach practical, one would like a good approximation of the set $h^{-1}(\mathcal{Y};w)$ that satisfies two properties:

- it should be easy to project onto, ideally a polytope or another simple convex set;
- it should be sufficiently accurate, especially near the boundary of $\mathcal{Y}$, where constraint satisfaction matters most.

A common assumption is that the admissible sets are polyhedral:

$$\mathcal{U} := \{u \in \mathbb{R}^m : \ Au \le b\}, \qquad \mathcal{Y} := \{y \in \mathbb{R}^p : \ Cy \le d\}.$$

In this case, the difficulty is entirely due to the nonlinear steady-state map. The set $\mathcal{Y}$ is simple in the output space, but its inverse image through h is not.

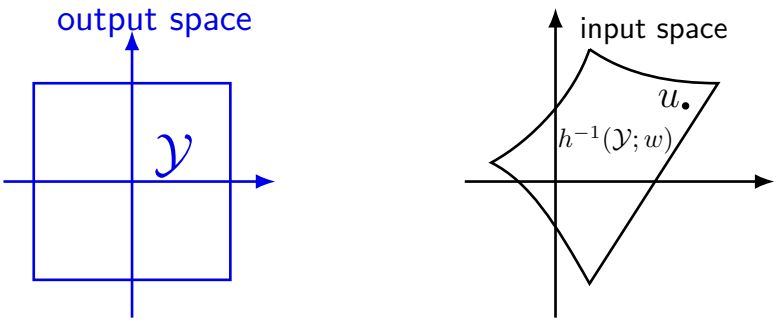

Figure 9: Geometry of the output constraint and its inverse image in the input space. A simple polyhedral constraint set $\mathcal{Y}$ in the output space induces, through the nonlinear map $h(u; w)$, a generally complicated feasible region $h^{-1}(\mathcal{Y}; w)$ in the input space. Intersecting this set with the input constraint set $\mathcal{U}$ yields the feasible set for projected gradient descent.

The natural way to obtain a tractable approximation is to linearize the steady-state map around the current input u. Using a first-order Taylor expansion, for a nearby point u' we have

$$h(u'; w) \approx h(u; w) + \nabla_u h(u; w)^\top (u' - u).$$

If the output constraint is

$$Cy \le d,$$

then substituting the first-order approximation gives

$$C\, h(u'; w) \le d \quad \Rightarrow \quad C\Big(h(u; w) + \nabla_u h(u; w)^\top (u' - u)\Big) \le d.$$

This defines a local first-order approximation of the inverse image $h^{-1}(\mathcal{Y}; w)$ around the current point u. Importantly, this approximation is affine in u', and therefore it produces a polyhedral local approximation of the feasible set in the input space.

So, instead of projecting onto the exact set $h^{-1}(\mathcal{Y}; w)$, which is difficult, one projects onto its linearized approximation. This preserves the main advantage of projected methods, namely explicit constraint handling, while replacing the hard nonlinear projection with a much simpler projection onto a local polytope.

This leads to the following interpretation. Projected gradient descent is conceptually attractive, because it tries to enforce feasibility directly. However, the exact feasible set in the input space is usually too complicated. The practical solution is therefore to replace the exact inverse image of the output constraints with a local first-order approximation based on the steady-state sensitivities

$$\nabla_u h(u; w).$$

Once again, the steady-state Jacobian is the central object: it provides the linear map that translates output constraints into approximate input constraints.

In summary:

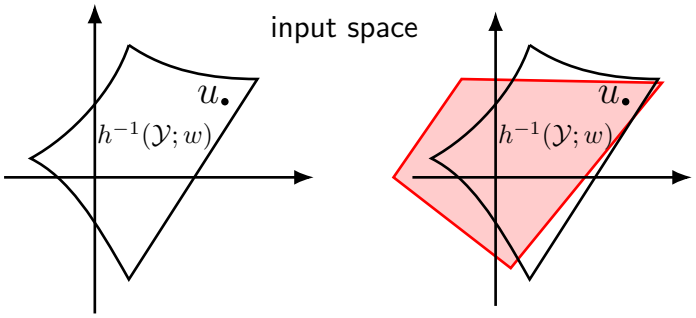

Figure 10: First-order approximation of the inverse image of the output constraint. Linearizing the steady-state map around the current input u transforms the nonlinear feasible set $h^{-1}(\mathcal{Y};w)$ into a local polyhedral approximation that is much easier to use in a projection step.

- the exact projected method would require projection onto

$$\tilde{\mathcal{U}} = \mathcal{U} \cap h^{-1}(\mathcal{Y};w),$$

  which is generally difficult;
- if $\mathcal{U}$ and $\mathcal{Y}$ are polyhedral, then a local linearization of h gives a polyhedral approximation of $h^{-1}(\mathcal{Y};w)$;
- this approximation makes projection computationally tractable and connects projected methods again to the steady-state sensitivity matrix $\nabla_u h(u;w)$.

4.11 Projected gradient flow via repeated quadratic programming

The discussion above suggests a practical refinement of projected gradient descent. The exact projection onto

$$\tilde{\mathcal{U}} = \mathcal{U} \cap h^{-1}(\mathcal{Y};w)$$

is typically hard, because the inverse image of the output constraint through the nonlinear steady-state map is not easy to describe explicitly. However, once a local first-order approximation of this inverse image is available, the projection step can be replaced by the solution of a simple quadratic program.

Assume that the input and output constraint sets are polyhedral:

$$\mathcal{U} := \{u \in \mathbb{R}^m : Au \le b\}, \qquad \mathcal{Y} := \{y \in \mathbb{R}^p : Cy \le d\}.$$

As seen above, around the current operating point u we can approximate the output constraint through the linearization

$$h(u';w) \approx h(u;w) + \nabla_u h(u;w)^\top (u' - u).$$

Therefore, the condition $y \in \mathcal{Y}$, namely

$$Cy \le d,$$

is locally approximated by

$$C\Big(h(u;w) + \nabla_u h(u;w)^\top (u' - u)\Big) \le d.$$

This suggests the following idea. Instead of computing the exact Euclidean projection of a gradient step onto the nonlinear feasible set, we search for a correction direction that is as close

as possible to the negative gradient while satisfying the linearized input and output constraints. In this way, each projected step is obtained by solving a quadratic program.

Let

$$g_k := \nabla_u \Phi(u_k, y_k) + \nabla_u h(u_k; w)^\top \nabla_y \Phi(u_k, y_k),$$

where y_k is the measured steady-state output corresponding to the current input u_k. The vector $-g_k$ is the descent direction that would be used in the unconstrained gradient method. Since constraints are present, we cannot move arbitrarily along this direction. We therefore compute a feasible correction δu_k as the solution of the quadratic program

$$\delta u_k := \arg\min_v \ \|v - (-g_k)\|^2$$

subject to

$$A(u_k + \alpha v) \le b,$$
$$C\Big(y_k + \alpha \nabla_u h(u_k; w)^\top v\Big) \le d.$$

The input is then updated according to

$$u_{k+1} = u_k + \alpha \delta u_k.$$

This construction has a clear interpretation. The optimization variable v is a candidate input increment. The quadratic objective forces v to remain as close as possible to the nominal negative gradient direction. The first constraint

$$A(u_k + \alpha v) \le b$$

imposes that the next input remains in the admissible input set $\mathcal{U}$. The second constraint

$$C\Big(y_k + \alpha \nabla_u h(u_k; w)^\top v\Big) \le d$$

is a first-order approximation of the output feasibility condition, centered at the current measurement y_k. So the quadratic program computes the feasible direction that best approximates the desired descent step.

This is why one may speak of *projected gradient flow via repeated quadratic programming*: the projection is no longer performed onto the exact nonlinear feasible set, but onto a local linearized approximation, and this approximate projection is obtained by solving a small QP at each iteration.

Several features of this construction are worth emphasizing.

First, the method uses the measured output y_k rather than the predicted value $h(u_k; w)$ in the constraint linearization. This is consistent with the feedback optimization philosophy: the plant measurement corrects the optimization step online and helps compensate model mismatch in the steady-state map itself.

Second, the Jacobian

$$\nabla_u h(u_k; w)$$

again plays the central role. It provides the local map from input variations to output variations, and therefore determines how output constraints are translated into local linear constraints on the input increment.

Third, the quadratic program is simple. Its cost is quadratic and convex, and its constraints are affine. Hence, under standard regularity conditions, it can be solved efficiently and repeatedly online. This makes the method much more practical than exact projection onto the nonlinear set $\tilde{\mathcal{U}}$.

In summary, the repeated-QP construction provides a tractable approximation of projected gradient descent:

- the unconstrained negative gradient gives the desired descent direction;
- the input and output constraints are enforced through local affine approximations;
- the feasible direction closest to the negative gradient is obtained from a quadratic program;
- the resulting update law preserves the feedback character of the method, because it is based on measured outputs and local steady-state sensitivities.

So the overall controller can be interpreted as a feedback optimization algorithm that repeatedly solves a local QP in order to generate a descent direction compatible with both input and output constraints.

4.12 Dual problem

Another important family of first-order methods for constrained optimization is based on *duality*. The idea is to keep the constraints explicitly, but to move them into the objective through Lagrange multipliers. This leads naturally to primal–dual algorithms and dual ascent methods, which are particularly well suited to feedback optimization.

To explain the idea, consider first the generic constrained optimization problem

$$\min_{x} \ \phi(x) \qquad \text{subject to} \qquad g(x) \leq 0,$$

where $g(x) \in \mathbb{R}^q$ collects the inequality constraints.

The associated Lagrangian is

$$\mathcal{L}(x, \lambda) := \phi(x) + \lambda^\top g(x),$$

where the multiplier vector satisfies

$$\lambda \geq 0.$$

The role of λ is to penalize constraint violations: whenever some component of $g(x)$ is positive, the corresponding multiplier increases the Lagrangian cost.

Under suitable convexity assumptions and constraint qualifications, strong duality holds. In that case, the original constrained problem can be written equivalently as

$$\min_{x:\, g(x)\leq 0} \phi(x) = \min_{x} \max_{\lambda \geq 0} \mathcal{L}(x, \lambda) = \max_{\lambda \geq 0} \min_{x} \mathcal{L}(x, \lambda).$$

This identity is the key reason why dual methods are useful: instead of solving the constrained problem directly, one may solve the saddle-point problem associated with the Lagrangian.

It is worth understanding the first equality. For a fixed x, consider

$$\max_{\lambda \geq 0} \mathcal{L}(x, \lambda) = \max_{\lambda \geq 0} \big(\phi(x) + \lambda^\top g(x)\big).$$

If $g(x) \leq 0$, then the maximum is attained at $\lambda = 0$, so

$$\max_{\lambda \geq 0} \mathcal{L}(x, \lambda) = \phi(x).$$

If instead some component of $g(x)$ is positive, then by increasing the corresponding multiplier one obtains

$$\max_{\lambda \geq 0} \mathcal{L}(x, \lambda) = +\infty.$$

So maximizing over the nonnegative multipliers has the effect of enforcing the constraint: infeasible points are assigned infinite cost, while feasible points retain their original objective value. In this sense, the Lagrangian reformulation replaces the constraint by an indicator-like mechanism.

This viewpoint is particularly useful in feedback optimization, because it leads to algorithms in which the primal variable performs a descent step, while the multiplier performs an ascent step. The multiplier then acts as an internal controller state that stores how strongly the output constraints are being violated.

4.13 Primal–dual iterations

The dual reformulation suggests natural iterative schemes for solving constrained optimization problems. Consider again the saddle-point problem

$$\max_{\lambda \geq 0} \min_{x} \mathcal{L}(x, \lambda), \qquad \mathcal{L}(x, \lambda) = \phi(x) + \lambda^\top g(x).$$

A first possibility is to apply gradient descent in the primal variable and gradient ascent in the dual variable. This yields the *primal descent / dual ascent* iteration

$$x_{k+1} = x_k - \alpha \nabla_x \mathcal{L}(x_k, \lambda_k),$$

$$\lambda_{k+1} = \lambda_k + \beta \nabla_\lambda \mathcal{L}(x_k, \lambda_k),$$

followed, if needed, by projection of λ_{k+1} onto the nonnegative orthant in order to preserve the condition $\lambda \geq 0$.

The logic is clear: one tries to decrease the Lagrangian with respect to the primal variable x, while increasing it with respect to the dual variable λ, since the saddle-point problem is a minimization in x and a maximization in λ.

A second possibility is to minimize the Lagrangian exactly with respect to the primal variable at each step, and then update only the multiplier by ascent. This gives the *dual ascent* scheme

$$x_k = \arg\min_{x} \mathcal{L}(x, \lambda_k),$$

$$\lambda_{k+1} = \lambda_k + \beta \nabla_\lambda \mathcal{L}(x_k, \lambda_k).$$

This method can be interpreted as a two-time-scale version of the primal–dual iteration: the primal variable reacts quickly and is assumed to be close to the minimizer of the current Lagrangian, while the multiplier evolves more slowly. In this sense, dual ascent is closely related to saddle-flow ideas with a slow dual dynamics.

A key observation is that

$$\nabla_\lambda \mathcal{L}(x, \lambda) = g(x),$$

because the Lagrangian is affine in the multiplier:

$$\mathcal{L}(x, \lambda) = \phi(x) + \lambda^\top g(x).$$

So the dual update is simply driven by the constraint violation. If the constraint is satisfied with margin, the multiplier tends not to grow; if the constraint is violated, the corresponding component of λ increases. This is exactly the mechanism by which the multiplier enforces feasibility.

4.14 Dual ascent for feedback optimization

We now specialize the primal–dual idea to the steady-state optimization problem of feedback optimization. Suppose that the output constraint set can be written as

$$\mathcal{Y} := \{y \in \mathbb{R}^p : \ g(y) \leq 0\},$$

where $g(y) \in \mathbb{R}^q$ collects the inequality constraints on the steady-state output.

Starting from the steady-state optimization problem

$$\min_{u,y} \ \Phi(u, y) \qquad \text{subject to} \qquad u \in \mathcal{U}, \quad y = h(u; w), \quad g(y) \leq 0,$$

we form the Lagrangian

$$\mathcal{L}(u, \lambda) := \Phi\big(u, h(u; w)\big) + \lambda^\top g\big(h(u; w)\big),$$

with

$$\lambda \geq 0.$$

Here the multiplier is associated with the output constraints. It increases the cost whenever the measured or predicted output violates the admissible set.

Applying the primal descent / dual ascent idea gives an iterative controller. The primal variable is the input u, while the dual variable is the multiplier λ. Using measurements for the output and a model only for the steady-state sensitivities, one obtains the update

$$u_{k+1} = u_k - \alpha\Big(\nabla_u \Phi(u_k, y_k) + \nabla_u h(u_k; w)^\top \nabla_y \Phi(u_k, y_k) + \nabla_u h(u_k; w)^\top \nabla g(y_k)^\top \lambda_k\Big),$$

and

$$\lambda_{k+1} = \Pi_{\geq 0}\big[\lambda_k + \beta g(y_k)\big].$$

Here:

- y_k is the measured plant output at iteration k;
- $\nabla_u h(u_k; w)$ is the steady-state sensitivity matrix provided by the model;
- $\Pi_{\geq 0}$ denotes projection onto the nonnegative orthant.

The interpretation of these equations is very important.

The primal update for u_k has the same structure as the gradient-based controllers discussed before, but now there is an extra term,

$$\nabla_u h(u_k; w)^\top \nabla g(y_k)^\top \lambda_k,$$

which comes from the Lagrange multiplier. This term pushes the input in a direction that reduces constraint violations. The stronger the current multiplier λ_k, the stronger the correction induced by the constraint.

The dual update is even more intuitive:

$$\lambda_{k+1} = \Pi_{\geq 0}\big[\lambda_k + \beta g(y_k)\big].$$

If some constraint is violated, then the corresponding component of $g(y_k)$ is positive, and so the corresponding multiplier increases. If instead the constraint is satisfied, the multiplier does not need to grow. The projection ensures that the multipliers remain nonnegative, as required by duality theory.

Once again, the implementation is remarkably light in terms of modeling requirements. The controller does not need a full dynamic model of the plant. It only needs:

- measurements of the current steady-state output y_k;
- the steady-state sensitivity matrix $\nabla_u h(u_k; w)$.

This is fully consistent with the feedback optimization philosophy developed so far.

Role of the multiplier as controller memory. An important conceptual question is: what is the role of the internal variable λ_k? The answer is that λ_k provides *memory* to the controller.

In plain gradient descent, the input update depends only on the current measurement and the current gradient information. In primal–dual control, instead, the multiplier stores information about past constraint violations. If a constraint has been active or violated for several iterations, the corresponding multiplier grows and keeps influencing the future control actions. In this sense, λ_k acts as an internal state that accumulates constraint-related information over time.

This is closely analogous to integral action in regulation problems. Just as an integral state accumulates tracking error in order to remove steady-state offset, the multiplier accumulates constraint violation in order to enforce feasibility. For this reason, the dual variable can be interpreted as a memory state of the feedback optimizer, dedicated specifically to the handling of output constraints.

So the primal–dual controller has a clear structure:

- the primal variable u_k moves in a descent direction for performance optimization;
- the dual variable λ_k integrates constraint violations;
- the interaction between the two produces a feedback law that seeks both optimality and feasibility.

This makes dual ascent and primal–dual iterations a very natural tool for online feedback optimization with constrained steady-state outputs.

4.15 An old friend: dual ascent and PI control

The primal–dual viewpoint is not only a mathematical construction. In an important special case, it recovers a very classical feedback controller.

Consider the linear steady-state relation

$$y = Hu + w,$$

where u is the manipulated input, y is the steady-state output, w is a constant exogenous signal, and r is the desired reference. Suppose we want to track the reference while keeping the input effort small. A natural optimization problem is

$$\begin{aligned} \min_{u,y} \quad & \frac{1}{2}\|u\|^2 + \frac{\rho}{2}\|Hu + w - r\|^2 \\ \text{s.t.} \quad & y = r, \\ & y = Hu + w. \end{aligned}$$

Equivalently, the constraint $y = r$ imposes exact tracking, while the term weighted by ρ plays the role of an augmented penalty on the steady-state mismatch. This is an augmented-Lagrangian formulation of the regulation problem.

The corresponding Lagrangian is

$$\mathcal{L}(u, \lambda) = \frac{1}{2}\|u\|^2 + \lambda^\top (Hu + w - r) + \frac{\rho}{2}\|Hu + w - r\|^2.$$

Here the multiplier λ is associated with the tracking constraint

$$Hu + w - r = 0.$$

Let us now apply the dual-ascent idea. As discussed above, one may combine:

- *primal minimization*, by imposing

$$\nabla_u \mathcal{L}(u_k, \lambda_k) = 0,$$

- *dual ascent*, by updating

$$\lambda_{k+1} = \lambda_k + \beta \nabla_\lambda \mathcal{L}(u_k, \lambda_k).$$

Since

$$\nabla_\lambda \mathcal{L}(u_k, \lambda_k) = Hu_k + w - r,$$

the dual update becomes

$$\lambda_{k+1} = \lambda_k + \beta(Hu_k + w - r).$$

If we denote the tracking error by

$$e_k := Hu_k + w - r,$$

this is simply

$$\lambda_{k+1} = \lambda_k + \beta e_k.$$

So the dual variable integrates the tracking error.

To compute the primal step, differentiate the Lagrangian with respect to u:

$$\nabla_u \mathcal{L}(u, \lambda) = u + H^\top \lambda + \rho H^\top (Hu + w - r).$$

Imposing

$$\nabla_u \mathcal{L}(u_k, \lambda_k) = 0$$

gives

$$u_k + H^\top \lambda_k + \rho H^\top (Hu_k + w - r) = 0,$$

that is,

$$u_k = -H^\top \lambda_k - \rho H^\top (Hu_k + w - r).$$

Up to sign conventions in the definition of the tracking error, this is exactly the structure of a proportional–integral controller:

- the term depending directly on the current error

$$H^\top (Hu_k + w - r)$$

  is a proportional action;

- the multiplier λ_k, updated by accumulation of the error, provides the integral action.

So dual ascent recovers a familiar control architecture: a PI controller appears as the primal–dual solution of a steady-state constrained optimization problem. This is a very useful interpretation. It shows that classical feedback structures can be understood as optimization algorithms, and conversely that dual variables in feedback optimization play the same role as integral states in regulation.

In this sense, the multiplier is indeed an "old friend": it is nothing but an integral memory that accumulates constraint or tracking error over time. This is why primal–dual methods often inherit the robustness properties usually associated with integral action, in particular the ability to remove constant steady-state offsets.

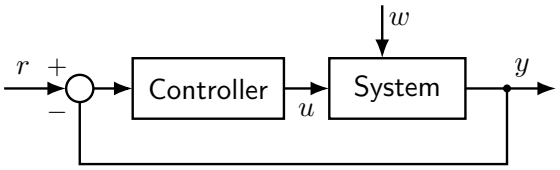

Figure 11: Dual-ascent interpretation of a classical feedback loop. For the linear steady-state relation $y = Hu + w$, primal minimization combined with dual ascent leads to a control law with a proportional term acting on the current tracking error and an integral term represented by the dual variable λ_k.

4.16 Design guidelines and tradeoffs

We can now summarize the main design options discussed so far for online feedback optimization. The three classes of methods considered in this chapter are:

- penalty-based gradient methods;
- projected-gradient methods;
- primal–dual or saddle-flow methods.

All three rely on the same basic ingredients:

- a fast and stable plant, so that the steady-state map is the relevant object;

- some model information in the form of steady-state sensitivities

$$\nabla_u h(u; w);$$

- output measurements y, which provide the feedback signal correcting the optimization step online.

What changes is how the constraints are handled, and this leads to different practical tradeoffs.

Penalty methods. Penalty methods incorporate the output constraint into the objective. Their main advantage is simplicity: one obtains a smooth unconstrained optimization law, often easy to implement and analyze. Moreover, constraint violation can be made arbitrarily small by choosing a sufficiently strong penalty. However, the violation is not removed exactly, only discouraged. In this sense, the behavior is essentially *proportional*: the correction grows with the violation, but there is no internal memory dedicated to enforcing exact feasibility. A second drawback is that a very steep penalty slows down the optimization dynamics and may lead to poor numerical conditioning or conservative steps.

Projected-gradient methods. Projected-gradient methods enforce feasibility more directly. By projecting the update onto an admissible set, they can guarantee *any-time feasibility*, at least with respect to the approximated constraint set used in the projection. This is very attractive when violating constraints is unacceptable. The price to pay is that one must solve a projection problem, which in practice often becomes a quadratic program. So feasibility is improved, but the controller is computationally heavier. In addition, the admissible stepsize is typically limited by the geometry of the projection and by the accuracy of the local approximation.

Saddle-flow or primal–dual methods. Primal–dual methods handle the constraints through dual variables. Their main advantage is that feasibility is enforced asymptotically through the multiplier dynamics. The dual variable acts as an integral memory, which is why these methods are often reminiscent of proportional–integral control. This makes them conceptually very appealing and often robust. On the other hand, because of the coupled primal and dual dynamics, oscillations may arise, especially if the gains are not tuned carefully or if the time scales of plant and optimizer are not sufficiently separated.

So the three approaches can be summarized as follows:

- **Penalty:** small violation allowed, simple structure, but steep penalties reduce speed;
- **Projected gradient:** feasibility enforced at every iteration, but requires repeated projection or quadratic programming;
- **Saddle flow / primal–dual:** feasibility obtained asymptotically through an integral-like mechanism, but oscillations may appear.

There is therefore no universally best design. The choice depends on the application requirements:

- if small temporary violations are acceptable and simplicity is important, penalty methods are attractive;
- if feasibility must be guaranteed at all times, projected-gradient methods are preferable;
- if one wants a dynamic mechanism that drives the system asymptotically to the constrained optimum, primal–dual methods are natural.

This comparison also clarifies the relation with classical control intuition:

- penalty methods behave in a mainly proportional way;
- primal–dual methods introduce an integral state through the multiplier;

- projected-gradient methods correspond instead to explicit feasibility correction through projection.

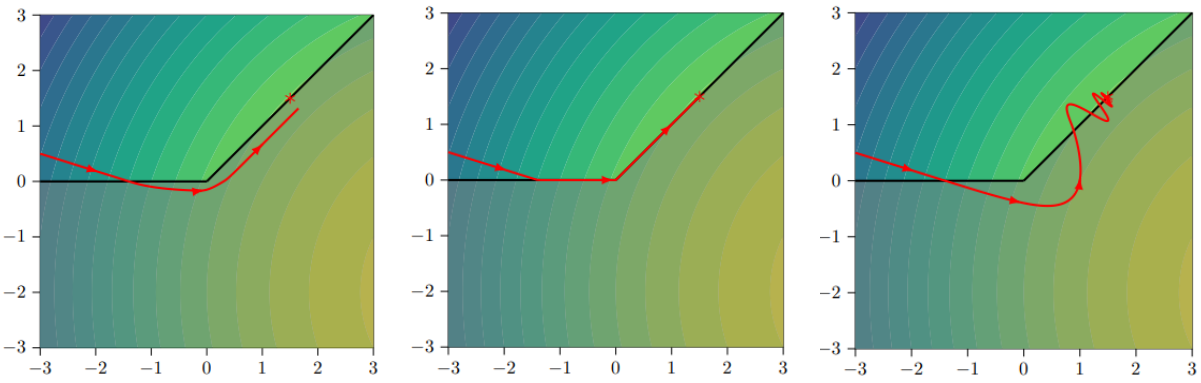

Figure 12: Qualitative comparison of the trajectories produced by penalty, projected-gradient, and saddle-flow methods. Penalty methods may tolerate small constraint violations, projected-gradient methods maintain feasibility through projection, and saddle-flow methods approach feasibility asymptotically but may exhibit oscillatory behavior.

In all cases, the same general message remains valid: feedback optimization does not require a full dynamical model of the plant, but it does require the right steady-state information. The central ingredients are the output measurements and the steady-state sensitivity matrix

$$\nabla_u h(u; w),$$

which together make it possible to embed optimization algorithms directly into the feedback loop.

4.17 Extremum seeking

So far, all feedback-optimization schemes relied on at least some approximate knowledge of the steady-state sensitivities

$$\nabla_u h(u; w),$$

or, equivalently, of the gradient of the performance map with respect to the input. In many applications, however, this information may be unavailable or too unreliable to be useful. This happens, for instance, when the plant is poorly modeled, when the operating conditions vary significantly over time, or when the steady-state map is known only through empirical performance charts.

This raises a natural question: can we optimize a steady-state performance index without any explicit knowledge of the gradient? Extremum-seeking control answers this question positively. It is a class of *zeroth-order* optimization methods, meaning that it seeks the optimizer by using only measurements of the cost or performance, without requiring an analytical model of its gradient.

The general setting is the following. We consider again a fast stable plant, and we associate to each constant input u a scalar performance metric

$$J(u),$$

which we want to minimize or maximize. The key difficulty is that the gradient

$$\nabla_u J(u)$$

is unknown. Extremum seeking overcomes this difficulty by injecting a small oscillatory perturbation into the input, observing the corresponding oscillatory component in the measured performance, and extracting from it an estimate of the local gradient.

This is why extremum seeking is often regarded as a learning-based feedback optimizer: it does not need a model of the steady-state sensitivities, because it estimates the necessary descent information online from measured data. In this sense, it complements the methods studied earlier in the chapter. When sensitivities are available, gradient, projected-gradient, or primal–dual methods are natural. When sensitivities are unknown, one may either estimate them or switch to a purely zeroth-order approach such as extremum seeking.

It is worth stressing one important robustness point. Even when the available sensitivity model is very inaccurate, the feedback-optimization schemes discussed previously may still work reasonably well. However, if one wants to avoid relying on such information altogether, extremum seeking is a natural alternative. This is one of the main motivations for introducing it at the end of the chapter.

Exercise. A useful fact to check is the following: both projected gradient descent and saddle-flow methods cannot have an equilibrium corresponding to an infeasible steady state

$$y \notin \mathcal{Y}.$$

This reinforces the idea that feedback optimization is structurally tied to feasibility. Extremum seeking, on the other hand, addresses a different issue: not the lack of feasibility mechanisms, but the lack of gradient information.

4.18 Extremum-seeking architecture

The simplest extremum-seeking idea can be explained for a scalar input. Suppose that the plant is fast and stable, and that the measured steady-state performance is $J(u)$. We want to optimize $J(u)$ without knowing its gradient.

The controller is built around four basic ingredients:

- an *exploration signal*, which perturbs the input with a small sinusoid;
- a *high-pass filter*, which removes the slowly varying offset in the measured performance;
- a *demodulation step*, which multiplies the filtered performance by a sinusoidal signal at the same frequency;
- a *low-pass filter* followed by an integrator, which extracts an average descent direction and updates the nominal input.

Denoting by $\hat{u}_k$ the slowly varying nominal input and by $\rho \sin(\omega k)$ the injected perturbation, the actual plant input is

$$u_k = \hat{u}_k + \rho \sin(\omega k).$$

So the plant is continuously excited around the nominal operating point. The purpose of this small oscillation is to probe the local slope of the performance function.

The measured performance $J(u_k)$ is then passed through a high-pass filter, which removes the average component and keeps the oscillatory part induced by the perturbation. This signal is subsequently multiplied by a sinusoid of the same frequency. This multiplication step is often called *demodulation*, because it extracts the component of the output that is correlated with the injected exploration signal. Finally, a low-pass filter removes the high-frequency terms, leaving only a slowly varying approximation of the gradient. The resulting signal is then integrated to update the nominal input $\hat{u}_k$ in the direction of improvement.

The exploration signal

$$\rho \sin(\omega k)$$

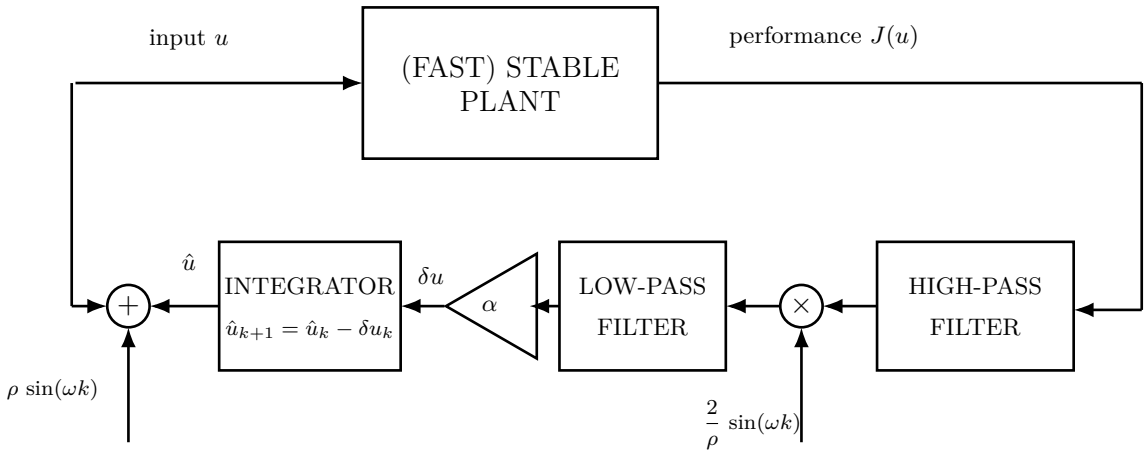

Figure 13: Basic extremum-seeking architecture. A small sinusoidal perturbation is added to the nominal input $\hat{u}_k$ in order to explore the local slope of the steady-state performance $J(u)$. The measured performance is high-pass filtered, demodulated, low-pass filtered, and finally integrated so as to generate an update of the nominal input.

should be thought of as a deliberate excitation used to reveal local gradient information. The demodulation signal, typically proportional to

$$\frac{2}{\rho} \sin(\omega k),$$

is chosen so that, after averaging, the product isolates the first-order term of the Taylor expansion of the performance function.

So the overall architecture can be interpreted as a feedback mechanism that reconstructs a gradient estimate from input–output oscillations, without ever requiring an explicit model of the plant or of the performance map.

4.19 Why extremum seeking behaves like gradient descent

The basic mechanism can be understood with a first-order Taylor expansion. Starting from

$$u_k = \hat{u}_k + \rho \sin(\omega k),$$

assume that ρ is small, so that the performance function can be expanded around the nominal input:

$$J(u_k) \approx J(\hat{u}_k) + \nabla_u J(\hat{u}_k)\, \rho \sin(\omega k).$$

The first term is a slowly varying offset, while the second is an oscillatory term whose amplitude is proportional to the unknown gradient.

After the high-pass filter, the offset is removed, and we are left approximately with

$$\nabla_u J(\hat{u}_k)\, \rho \sin(\omega k).$$

The next step is demodulation. Multiplying by

$$\frac{2}{\rho} \sin(\omega k)$$

gives

$$\nabla_u J(\hat{u}_k)\, 2 \sin^2(\omega k).$$

Using the identity

$$2 \sin^2(\omega k) = 1 - \cos(2\omega k),$$

we see that this signal consists of a constant part,

$$\nabla_u J(\hat{u}_k),$$

plus a fast oscillation at frequency 2ω. The low-pass filter removes the oscillatory component and keeps only the average value. Therefore, after the low-pass filter, the signal is approximately

$$\nabla_u J(\hat{u}_k).$$

If this signal is then multiplied by a gain α, we obtain the update direction

$$\delta u_k = \alpha \nabla_u J(\hat{u}_k).$$

Finally, the integrator implements

$$\hat{u}_{k+1} = \hat{u}_k - \delta u_k,$$

hence

$$\hat{u}_{k+1} = \hat{u}_k - \alpha \nabla_u J(\hat{u}_k).$$

But this is exactly the gradient-descent iteration.

So, at an intuitive level, extremum seeking is nothing but gradient descent in disguise:

- the exploration signal reveals the local slope of the performance map;
- the filtering and demodulation reconstruct the gradient;
- the integrator performs the actual descent step.

Of course, this derivation is only a sketch. A rigorous analysis requires averaging arguments, time-scale separation, and assumptions on the filters and on the plant dynamics. But the basic idea is exactly the one above: extremum seeking creates its own gradient information through oscillatory probing.

4.20 Applications of extremum seeking

Extremum seeking is especially attractive in applications with the following features:

- the plant is fast and stable at the time scale of the optimizer;
- the steady-state performance map is poorly known or strongly parameter dependent;
- the optimization problem is low dimensional;
- small oscillatory perturbations of the input are acceptable.

A classical example is photovoltaic maximum-power-point tracking. In a photovoltaic system, the active power generated by the panel depends on the operating voltage, and the power–voltage curve changes significantly with solar irradiance and temperature. Therefore, the location of the optimal voltage is not fixed and may vary continuously during operation. Moreover, an accurate model of the full dependence on the environmental variables may be difficult to maintain online.

In this situation, extremum seeking is very natural. By slightly perturbing the voltage and measuring the resulting power, the controller can estimate whether the current operating point is to the left or to the right of the maximum, and then move the nominal voltage in the correct direction. In this way, the controller tracks the operating voltage that maximizes active power generation without requiring an explicit gradient model.

This is a particularly good use case because:

- the optimum varies with exogenous conditions;
- the model is poor or uncertain;
- the control input is low dimensional;

- the actuator cost of a small oscillatory probing signal is acceptable.

Other relevant applications include internal combustion engines, chemical processes, compressor systems, and meta-optimization problems such as tuning controller parameters. In all these cases, extremum seeking is valuable when the optimizer can rely on measured performance only, while the gradient or the steady-state sensitivity is unavailable.

Final perspective. This closes the chapter with an important conceptual message. Feedback optimization can be organized along a spectrum:

- if reliable steady-state sensitivities are available, one can use gradient, projected-gradient, or primal–dual schemes;
- if sensitivities are inaccurate, one may still rely on robust feedback optimization based on approximate models;
- if sensitivities are completely unknown, one can move to learning or zeroth-order methods such as extremum seeking.

So extremum seeking is not a separate topic disconnected from the rest. It is the natural endpoint of the feedback-optimization viewpoint: when model-based gradient information disappears, the controller can still recover optimization directions directly from feedback measurements.

5 System Identification

So far, the control methods that we have studied, in particular LQR and Model Predictive Control, have relied on the availability of a state-space model of the plant. In discrete time, this is typically written as

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t,$$

or, more generally, with a nonlinear update map. This representation is extremely convenient for control design, because it provides a Markovian description of the system: once the current state is known, the future evolution depends only on the present state and on the input that will be applied. In this sense, the state is the variable that summarizes all the information from the past that is relevant for predicting the future.

Having such a model, however, means having a very detailed description of the plant. One must know the parameters that govern the state update, the way in which the input affects the state evolution, and the way in which the internal state is mapped into the measured outputs. Even more fundamentally, one must decide what the state actually is, that is, which variables are sufficient to obtain a Markovian model. This is already a nontrivial modeling choice, especially when the internal physical variables are not directly measurable.

A first important observation is that a state-space representation is not unique. Different internal coordinates may describe exactly the same external input-output behavior. Indeed, if we introduce a new state variable w_t through an invertible linear change of coordinates

$$x_t = Tw_t,$$

with T nonsingular, then the system can be equivalently written as

$$w_{t+1} = T^{-1}ATw_t + T^{-1}Bu_t, \qquad y_t = CTw_t.$$

Although the matrices appearing in the model have changed, the input-output trajectories remain exactly the same. Therefore, the state should not be interpreted as a uniquely defined physical object, but rather as an internal variable that makes prediction and control possible. What really matters from the external viewpoint is the input-output behavior of the system.

This point is one of the main motivations for system identification. If different state-space models can generate the same measured behavior, then reconstructing a model from data cannot be understood as recovering the "true" internal realization in an absolute sense. Instead, the goal is to build a model that reproduces the observed dynamics well enough for the intended task. In our context, this task is control. Therefore, the identified model should be sufficiently accurate to predict the future effect of the control inputs and to support the synthesis of model-based controllers.

System identification addresses exactly this problem. Rather than deriving the model entirely from first principles, we use measured input-output data to infer a dynamical description of the plant. This is especially useful when a first-principles model is unavailable, too expensive to derive, or too inaccurate for control purposes. The identified model may be a state-space realization, an input-output model, or another predictive representation, depending on the assumptions and on the control architecture of interest.

From this viewpoint, identification can be seen as the bridge between data and model-based control. On the one hand, advanced controllers such as LQR and MPC require a predictive model. On the other hand, the model need not be known a priori: it can be estimated from experiments. The central question of this chapter is therefore how to extract, from measured data, a model that captures the relevant dynamical behavior of the plant and can be reliably used for prediction and control.

A natural way to move from an internal state-space description to an external input-output description is to write the output as a function of the initial condition and of the past inputs.

For the discrete-time linear system

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t,$$

repeated substitution gives

$$y_t = CA^t x_0 + \sum_{j=0}^{t-1} CA^{t-1-j} B\, u_j.$$

This expression clearly separates two contributions. The term $CA^t x_0$ is the response generated by the initial state, while the summation describes how past inputs are propagated to the output through the dynamics. The latter has the form of a discrete-time convolution and shows that, for linear time-invariant systems, the full input-output behavior is encoded in the sequence of matrices

$$CA^{t-1}B, \qquad t > 0.$$

In the SISO case, where both $u_t \in \mathbb{R}$ and $y_t \in \mathbb{R}$, this sequence reduces to a scalar sequence

$$h_t = CA^{t-1}B, \qquad t > 0,$$

which is the impulse response of the system. If the initial condition is zero, then the output produced by an arbitrary input sequence is completely determined by the convolution of the input with $\{h_t\}_{t\geq 1}$. In this sense, the impulse response is a complete representation of the zero-state input-output map of the plant. It contains exactly the information needed to predict future outputs from past inputs, without explicitly referring to the internal state coordinates. This is especially important in identification, because the state itself is not directly unique, whereas the input-output behavior is the physically observable object.

The same idea extends immediately to MIMO systems. If the plant has m inputs and p outputs, then each coefficient

$$H_t = CA^{t-1}B \in \mathbb{R}^{p\times m}, \qquad t > 0,$$

is a matrix. Its entry in position (i, j) represents the response of output i to a unit impulse applied on input j. The sequence $\{H_t\}_{t\geq 1}$ is called the sequence of Markov parameters of the system. These parameters generalize the impulse response and provide a compact description of the input-output behavior of the linear system. Once they are known, the output can be reconstructed from the past inputs through matrix convolution.

This viewpoint is also attractive from an experimental perspective. On a real plant, the Markov parameters can be estimated directly from data by exciting the system and measuring the corresponding outputs. In the ideal noise-free case, one can initialize the plant at zero and apply a unit impulse to one input channel at a time, while keeping the others equal to zero. For a MIMO system, this means that m experiments are sufficient in principle: each experiment isolates one column of the matrices H_t, and by repeating the procedure for all inputs one reconstructs the full sequence of Markov parameters. In practice, however, perfect impulses cannot be generated and measurements are affected by noise, disturbances, and possible offsets in the initial condition. For this reason, identification methods usually rely on richer input signals and estimate the Markov parameters from batches of input-output data, rather than from a single idealized impulse experiment.

The key message is that, before identifying a state-space realization, one may already identify the external behavior of the plant through its impulse response, or equivalently through its Markov parameters. This provides the first bridge between measured data and predictive models, and it motivates the identification techniques developed in the rest of this chapter.

5.1 Controllability, observability, and minimal realizations

When the goal is to identify a state-space model from input-output data, not every realization is equally meaningful. Since the state is not unique, one can always introduce additional coordinates that do not change the external behavior of the system. For identification and control, however, we are interested in realizations in which the state has an actual dynamical role. This naturally leads to the notions of controllability and observability, which characterize whether the state can be influenced by the input and reconstructed from the output.

For the discrete-time linear system

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t,$$

controllability concerns the possibility of steering the system from a given initial condition to a desired target state. To fix ideas, let us start from $x_0 = 0$. After one step, the state is

$$x_1 = Bu_0,$$

so the set of states reachable in one step is exactly the image of B. After two steps,

$$x_2 = ABu_0 + Bu_1,$$

hence the reachable set is the image of the matrix $[\,B \;\; AB\,]$. Continuing in the same way, after n steps the state belongs to the image of

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix},$$

which is called the controllability matrix. Therefore, the system is controllable precisely when every state in $\mathbb{R}^n$ can be reached, that is, when

$$\operatorname{rank}(\mathcal{C}) = n.$$

This condition means that the columns generated by repeatedly propagating the input directions through the dynamics span the whole state space. It is worth stressing that n steps are enough to decide controllability: if the full state space has not been reached by then, taking more steps cannot generate new independent directions.

The controllability matrix also provides a constructive way to compute a control sequence that reaches a desired state. Indeed, for a target $\bar{x}$ reached in n steps, one can write

$$x_n = \mathcal{C} \begin{bmatrix} u_{n-1} \\ \vdots \\ u_0 \end{bmatrix}.$$

If $\mathcal{C}$ is square and invertible, which is possible in particular in the scalar-input case, the input sequence is obtained directly by inversion. In the more general case, $\mathcal{C}$ is not square, and the problem may admit multiple solutions. A standard choice is then the minimum-norm solution, which can be computed through the pseudoinverse.

Dual to controllability is observability. While controllability asks whether the input can excite all internal directions, observability asks whether the effect of the initial state can be detected from the output. Starting again from the linear model, at time $t = 0$ one has

$$y_0 = Cx_0,$$

so any initial state in the kernel of C is invisible at the output at that instant. After one step,

$$y_1 = CAx_0,$$

hence a state that belongs simultaneously to the kernels of C and CA remains indistinguishable over the first two output samples. Proceeding recursively, after n steps the invisible states are those in the kernel of

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix},$$

which is called the observability matrix. The system is observable if and only if

$$\operatorname{rank}(\mathcal{O}) = n.$$

Equivalently, distinct initial states always generate distinct output trajectories over a finite observation window. Also here, n steps are sufficient: if a state cannot be distinguished from the output sequence of length n, it will remain unobservable even if more measurements are collected.

As in the controllability case, observability has a constructive interpretation. Given an output sequence $y_0, \dots, y_{n-1}$, one can write

$$\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix} = \mathcal{O} x_0.$$

If $\mathcal{O}$ is invertible, the initial state is uniquely determined. More generally, when $\mathcal{O}$ has full column rank, the state can still be reconstructed uniquely, for instance through a least-squares inversion based on the pseudoinverse.

These two notions are central in system identification because they single out the realizations that are meaningful from input-output data. If a realization contains uncontrollable states, part of the state cannot be influenced by the input and is therefore irrelevant for prediction and control. If it contains unobservable states, part of the state has no effect on the measured output and therefore cannot be inferred from data. In both cases, the realization contains redundant internal coordinates. For this reason, in identification we focus on realizations that are both controllable and observable. Such realizations are called minimal, because they do not contain superfluous states and represent the input-output behavior with the smallest possible state dimension.

A simple example illustrates the point. Consider a plant whose output is a moving average of the past three inputs,

$$y_t = a_1 u_{t-1} + a_2 u_{t-2} + a_3 u_{t-3}.$$

A natural state is obtained by storing delayed copies of the input:

$$x_t = \begin{bmatrix} u_{t-1} \\ u_{t-2} \\ u_{t-3} \end{bmatrix}.$$

With this choice, the dynamics become

$$x_{t+1} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} x_t + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} u_t, \qquad y_t = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} x_t.$$

This realization has an immediate interpretation: the state is simply a memory of the past input values needed to compute the current output.

The controllability matrix of this realization is

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix} = I,$$

so the system is controllable. This is consistent with the physical meaning of the state, since the input directly writes the first memory cell and, through the shift dynamics, can generate any delayed input profile. The observability matrix is

$$\mathcal{O} = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_2 & a_3 & 0 \\ a_3 & 0 & 0 \end{bmatrix}.$$

Hence observability depends on the coefficients a_1, a_2, a_3. When this matrix has full rank, the three stored delays all leave a visible signature in the output and the realization is minimal. If instead the rank is smaller, then some component of the chosen state does not affect the output independently, which means that a lower-order realization exists.

This example clarifies why controllability and observability are so important in identification. Starting from input-output data, one does not want to recover an arbitrary realization, but rather a realization whose state is both reachable from the input and visible from the output. Only in this case does the state provide a well-posed internal representation of the external behavior of the plant, and only in this case can we hope to obtain a compact and useful model for control design.

5.2 Realization from data and the Ho–Kalman construction

The discussion so far has shown that the input-output behavior of a linear time-invariant system is completely described by its Markov parameters

$$\{CB,\ CAB,\ CA^2B,\ \dots\}.$$

This naturally leads to the realization problem: if these parameters are known from experiments, how can one reconstruct a state-space model

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t$$

that generates them? More generally, given generic input-output data, how can one construct a realization A, B, C consistent with the observed dynamics? The goal is not to recover a unique internal state, since different state-space models may have the same external behavior, but rather to compute a minimal realization, that is, one that is both controllable and observable.

The key object in this construction is the Hankel matrix built from the Markov parameters. If we define the observability and controllability matrices of a minimal realization as

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}, \qquad \mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix},$$

then their product is

$$\mathcal{H} = \mathcal{O}\mathcal{C}.$$

Expanding this product shows that every block of $\mathcal{H}$ is one of the Markov parameters:

$$\mathcal{H} = \begin{bmatrix} CB & CAB & CA^2B & \cdots & CA^{n-1}B \\ CAB & CA^2B & CA^3B & \cdots & CA^nB \\ CA^2B & CA^3B & CA^4B & \cdots & CA^{n+1}B \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ CA^{n-1}B & CA^nB & CA^{n+1}B & \cdots & CA^{2n-2}B \end{bmatrix}.$$

Thus the Hankel matrix can be assembled directly from experimental data as soon as enough Markov parameters are available. This is the crucial bridge between input-output measurements and state-space realizations.

To make the structure visually apparent, it is often helpful to highlight repeated Markov parameters along the anti-diagonals. For example, one may write

$$\mathcal{H} = \begin{bmatrix} CB & CAB & CA^2B & \cdots & CA^{n-1}B \\ CAB & CA^2B & CA^3B & \cdots & CA^nB \\ CA^2B & CA^3B & CA^4B & \cdots & CA^{n+1}B \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ CA^{n-1}B & CA^nB & CA^{n+1}B & \cdots & CA^{2n-2}B \end{bmatrix}.$$

The repeated colors emphasize that each anti-diagonal contains the same Markov parameter. This is exactly the signature of a Hankel structure: the matrix is constant along anti-diagonals, and those constants are the impulse-response coefficients of the system.

The Ho–Kalman algorithm uses this structure in three conceptual steps. First, one constructs the Hankel matrix from the measured Markov parameters. Second, one factorizes the Hankel matrix as

$$\mathcal{H} = \mathcal{O}\mathcal{C}.$$

Third, one extracts a realization A, B, C from the factors $\mathcal{O}$ and $\mathcal{C}$.

At first sight, the first step raises a basic difficulty: how many Markov parameters are needed? The answer depends on the order n of the minimal realization, which is unknown a priori. However, if the realization is minimal, then both $\mathcal{O}$ and $\mathcal{C}$ have rank n, and therefore the Hankel matrix also has rank n. This means that, in principle, one can enlarge the Hankel matrix progressively until its rank stops increasing. That plateau reveals the order of the system. In noise-free settings this idea works exactly; in practice, with noisy data, the same principle survives in approximate form and the effective rank is inferred numerically.

This rank-based interpretation is easy to understand on simple examples. A first-order low-pass filter has state-space model

$$x_{t+1} = ax_t + u_t, \qquad y_t = x_t,$$

so its Markov parameters are $1, a, a^2, \ldots$, and the corresponding Hankel matrix has rank one. By contrast, a frictionless cart model written as

$$v_{t+1} = v_t + u_t, \qquad z_{t+1} = z_t + v_t, \qquad y_t = z_t$$

has a two-dimensional state, and the Hankel matrix reflects this higher order through rank two. In this way, the rank of the Hankel matrix captures the intrinsic dimension of the minimal realization.

The second step of the Ho–Kalman method is the factorization

$$\mathcal{H} = \mathcal{O}\mathcal{C}.$$

An important conceptual point is that this factorization is not unique, but this is not a problem. Indeed, nonuniqueness of the factorization simply reflects nonuniqueness of the state coordinates. If we apply a change of basis

$$\tilde{A} = T^{-1}AT, \qquad \tilde{B} = T^{-1}B, \qquad \tilde{C} = CT,$$

with T invertible, then the corresponding observability and controllability matrices become

$$\tilde{\mathcal{O}} = \begin{bmatrix} \tilde{C} \\ \tilde{C}\tilde{A} \\ \tilde{C}\tilde{A}^2 \\ \vdots \end{bmatrix} = \begin{bmatrix} CT \\ CTAT^{-1}T \\ CTA^2T^{-1}T \\ \vdots \end{bmatrix} = \mathcal{O}T,$$

and similarly

$$\tilde{\mathcal{C}} = T^{-1}\mathcal{C}.$$

Therefore,

$$\tilde{\mathcal{O}}\tilde{\mathcal{C}} = \mathcal{O}TT^{-1}\mathcal{C} = \mathcal{O}\mathcal{C} = \mathcal{H}.$$

So different factorizations of $\mathcal{H}$ correspond exactly to different coordinate representations of the same input-output system. The factorization does not identify a unique realization, but it identifies the system up to similarity transformation, which is the correct notion of uniqueness for state-space models.

Because of this, the factorization step leaves some freedom. In principle, one could take trivial choices such as $\mathcal{O} = I$ and $\mathcal{C} = \mathcal{H}$, or $\mathcal{O} = \mathcal{H}$ and $\mathcal{C} = I$, but these choices are not useful numerically and do not directly reveal a well-scaled state representation. In practice, one prefers factorizations based on the singular value decomposition, because they expose the rank of the Hankel matrix and provide numerically robust coordinates for the state.

Once a factorization $\mathcal{H} = \mathcal{O}\mathcal{C}$ has been obtained, the realization matrices are recovered from the shift structure of $\mathcal{O}$ and $\mathcal{C}$. The matrix C is simply the first block row of $\mathcal{O}$, while B is the first block column of $\mathcal{C}$. The state transition matrix A is then determined by exploiting the fact that consecutive block rows of $\mathcal{O}$ differ by multiplication by A, or equivalently that consecutive block columns of $\mathcal{C}$ differ by the same shift. In this way, the whole state-space realization is reconstructed from the Hankel matrix, and therefore from measured input-output behavior alone.

The significance of the Ho–Kalman construction is that it turns impulse-response data into a minimal state-space model through linear-algebraic operations. It provides one of the cleanest answers to the realization problem: first identify the Markov parameters, then organize them into a Hankel matrix, then factorize this matrix to uncover the underlying controllable and observable state structure.

5.3 SVD-based Ho–Kalman realization

The key missing ingredient in the Ho–Kalman construction is a numerically reliable way to factorize the Hankel matrix. This is provided by the singular value decomposition. If $H \in \mathbb{R}^{m\times n}$ has rank r, then it can be written as

$$H = U\Sigma V^\top,$$

where $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal matrices, while $\Sigma \in \mathbb{R}^{m\times n}$ is diagonal in the rectangular sense, with nonnegative singular values on the diagonal. If the rank of H is r, then only the first r singular values are nonzero, and the decomposition can be partitioned as

$$H = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1 & V_2 \end{bmatrix}^\top = U_1\Sigma_1 V_1^\top,$$

where $\Sigma_1 \in \mathbb{R}^{r\times r}$ contains the nonzero singular values. This decomposition is one of the most useful tools in numerical linear algebra: it is computationally efficient, numerically robust, and it reveals directly the rank and the dominant subspaces of the matrix. In particular, the columns of U_1 span the image of H, while the dimension of Σ_1 identifies the rank.

This is exactly the type of factorization we need for the Hankel matrix. Instead of building a square Hankel matrix of unknown dimension, it is convenient to introduce an extended Hankel matrix

$$\mathcal{H}_{k,\ell} = \begin{bmatrix} G_1 & G_2 & \cdots & G_\ell \\ G_2 & G_3 & \cdots & G_{\ell+1} \\ G_3 & G_4 & \cdots & G_{\ell+2} \\ \vdots & \vdots & \ddots & \vdots \\ G_k & G_{k+1} & \cdots & G_{k+\ell-1} \end{bmatrix},$$

where $G_t = CA^{t-1}B$ denotes the t-th Markov parameter. Here k and ℓ are chosen larger than the unknown system order n. If the data are generated by an n-dimensional minimal realization, then this extended Hankel matrix factors as

$$\mathcal{H}_{k,\ell} = \mathcal{O}_k \mathcal{C}_\ell,$$

with

$$\mathcal{O}_k = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{k-1} \end{bmatrix}, \qquad \mathcal{C}_\ell = \begin{bmatrix} B & AB & A^2B & \cdots & A^{\ell-1}B \end{bmatrix}.$$

These are the extended observability and controllability matrices. As soon as $k \geq n$ and $\ell \geq n$, both matrices have rank n, and therefore $\mathcal{H}_{k,\ell}$ also has rank n. This is the fundamental reason why one chooses k and ℓ larger than the system order: once the horizon is long enough, the rank of the Hankel matrix reveals the true dimension of the minimal realization.

At this point, the SVD gives a natural and balanced factorization of $\mathcal{H}_{k,\ell}$. If

$$\mathcal{H}_{k,\ell} = U_1 \Sigma_1 V_1^\top,$$

where Σ_1 contains the nonzero singular values, then the order of the system is simply

$$n = \operatorname{rank}(\mathcal{H}_{k,\ell}) = \dim(\Sigma_1).$$

Moreover, we may decompose the Hankel matrix as

$$\mathcal{H}_{k,\ell} = \underbrace{U_1 \Sigma_1^{1/2}}_{\mathcal{O}_k} \underbrace{\Sigma_1^{1/2} V_1^\top}_{\mathcal{C}_\ell}.$$

This choice is especially convenient because it distributes the scaling symmetrically between the observability and controllability factors. In this way we obtain numerical representatives of the extended observability and controllability matrices:

$$\mathcal{O}_k = U_1 \Sigma_1^{1/2}, \qquad \mathcal{C}_\ell = \Sigma_1^{1/2} V_1^\top.$$

From these matrices, C and B are obtained immediately. Indeed, C is the first block row of $\mathcal{O}_k$, while B is the first block column of $\mathcal{C}_\ell$. Visually, one can emphasize this as

$$\mathcal{O}_k = \begin{bmatrix} \boxed{C} \\ CA \\ CA^2 \\ \vdots \\ CA^{k-1} \end{bmatrix}, \qquad \mathcal{C}_\ell = \begin{bmatrix} \boxed{B} & AB & A^2B & \cdots & A^{\ell-1}B \end{bmatrix}.$$

Thus the first visible block of the observability factor gives the output matrix, and the first visible block of the controllability factor gives the input matrix.

To recover the state transition matrix A, one introduces the shifted Hankel matrix

$$\mathcal{H}_{k,\ell}^{\uparrow} = \begin{bmatrix} G_2 & G_3 & \cdots & G_{\ell+1} \\ G_3 & G_4 & \cdots & G_{\ell+2} \\ G_4 & G_5 & \cdots & G_{\ell+3} \\ \vdots & \vdots & \ddots & \vdots \\ G_{k+1} & G_{k+2} & \cdots & G_{k+\ell} \end{bmatrix}.$$

By construction, this matrix satisfies

$$\mathcal{H}_{k,\ell}^{\uparrow} = \begin{bmatrix} CAB & CA^2B & CA^3B & \cdots \\ CA^2B & CA^3B & CA^4B & \cdots \\ CA^3B & CA^4B & CA^5B & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} = \mathcal{O}_k A \mathcal{C}_\ell.$$

Once $\mathcal{O}_k$ and $\mathcal{C}_\ell$ are known, this relation can be inverted in the least-squares sense, yielding

$$A = \mathcal{O}_k^{\dagger} \mathcal{H}_{k,\ell}^{\uparrow} \mathcal{C}_\ell^{\dagger},$$

where † denotes the pseudoinverse. Therefore the full realization (A, B, C) is reconstructed directly from the measured Markov parameters.

Putting everything together, the Ho–Kalman procedure can be summarized as follows. First, one collects impulse-response data and estimates the Markov parameters of the system. Then one chooses k, ℓ sufficiently large and builds the extended Hankel matrix $\mathcal{H}_{k,\ell}$ together with its shifted version $\mathcal{H}_{k,\ell}^{\uparrow}$. Next, one computes the SVD of $\mathcal{H}_{k,\ell}$, uses the nonzero singular values to determine the system order n, and defines the extended observability and controllability matrices through the balanced factorization

$$\mathcal{O}_k = U_1 \Sigma_1^{1/2}, \qquad \mathcal{C}_\ell = \Sigma_1^{1/2} V_1^{\top}.$$

Finally, one extracts

$$C = \text{first block row of } \mathcal{O}_k, \qquad B = \text{first block column of } \mathcal{C}_\ell,$$

and computes

$$A = \mathcal{O}_k^{\dagger} \mathcal{H}_{k,\ell}^{\uparrow} \mathcal{C}_\ell^{\dagger}.$$

The main message is that, even when a state-space model is not available a priori, it can be reconstructed from data. In this sense, the data themselves contain the model information needed for prediction and control. Once a realization has been identified, the model-based control methods developed in the previous chapters, such as LQR and MPC, can be applied to the identified plant exactly as if the model had been known from first principles.

At the same time, it is important to keep in mind the assumptions underlying this construction. The derivation assumes that the plant is an exact finite-dimensional linear time-invariant system and that the Markov parameters are known without noise. Under these ideal conditions, the rank of the Hankel matrix is sharp, the realization is exact, and the Ho–Kalman algorithm recovers a minimal state-space model up to a change of coordinates. Real identification problems are usually more challenging: measurements are noisy, data are finite, and the true dynamics may be nonlinear or only approximately finite-dimensional. Therefore, the material presented here should be understood as a first and fundamental example of system identification, rather than as a complete treatment of the subject. More advanced settings include noisy and stochastic data, missing measurements, prior structural information, nonlinear models, reduced-order representations, and both parametric and nonparametric identification methods. Still, the ideas developed in this chapter remain central: organize data into structured matrices, reveal the intrinsic dimension through linear algebra, and reconstruct a model that captures the dynamical behavior relevant for control.

6 Data-driven LQR

A Convex optimization

Convex optimization appears repeatedly in computational control. In unconstrained LQR, the stage optimization is a convex quadratic problem and can be solved in closed form. In constrained MPC, the online problem is typically a convex quadratic program. In feedback optimization, first-order methods, projections, and dual updates are all built on convexity assumptions. For this reason, it is useful to collect in one place the basic facts on convex sets, convex functions, and optimality conditions that make these methods computationally tractable.

At a high level, modern computers are particularly effective at two things: linear algebra and iteration. Linear algebra makes it possible to manipulate vectors and matrices efficiently, solve linear systems, and exploit structure in large-scale problems. Iteration makes it possible to repeat simple update rules until convergence. Numerical optimization sits exactly at the intersection of these two capabilities. This is one of the main reasons why optimization-based control can be implemented in real time when the underlying problem has the right structure, and convexity is precisely the property that provides such structure.

We consider the generic optimization problem

$$\begin{aligned} \min_{x \in X} \quad & f(x) \\ \text{s.t.} \quad & g(x) \leq 0, \\ & h(x) = 0, \end{aligned} \tag{3}$$

where $x \in X \subseteq \mathbb{R}^n$ is the decision variable, $f : \mathbb{R}^n \to \mathbb{R}$ is the cost function, $g : \mathbb{R}^n \to \mathbb{R}^m$ collects inequality constraints, and $h : \mathbb{R}^n \to \mathbb{R}^p$ collects equality constraints.
This formulation is general enough to include many control problems. For example, the finite-horizon MPC problem can be written in this form after stacking all predicted states and inputs into a single vector. The key question is then not only whether a minimizer exists, but whether it can be computed efficiently and reliably online. Convexity provides a positive answer in a large class of relevant cases.

A.1 Convex sets and convex functions

We start from the geometric notion of convexity.
A set $X \subseteq \mathbb{R}^n$ is called *convex* if for every $x', x'' \in X$ and every $t \in [0, 1]$,

$$tx' + (1 - t)x'' \in X.$$

In words, whenever two points belong to the set, the whole line segment joining them also belongs to the set.
A function $\phi : \mathbb{R}^n \to \mathbb{R}$ is called *convex* on a convex domain if for every x', x'' in its domain and every $t \in [0, 1]$,

$$\phi(tx' + (1 - t)x'') \leq t\phi(x') + (1 - t)\phi(x'').$$

This means that the graph of the function lies below the straight line joining the function values at the two endpoints.

These two definitions are closely related. A fundamental fact is that if g is convex, then every sublevel set

$$\{x \mid g(x) \leq c\}$$

is convex. Indeed, if x' and x'' both satisfy $g(x) \leq c$, then by convexity

$$g(tx' + (1 - t)x'') \leq tg(x') + (1 - t)g(x'') \leq tc + (1 - t)c = c,$$

so the convex combination remains in the sublevel set.

The converse, however, is false in general: it is possible for a function to have convex sublevel sets without being convex. Therefore, convexity of the function is a stronger property than convexity of one particular feasible region.

Several sets that appear naturally in control and optimization are convex.
A *halfspace* is a set of the form

$$\{x \mid a^\top x \leq b\},$$

where $a \in \mathbb{R}^n$ and $b \in \mathbb{R}$. Geometrically, a halfspace is one side of a hyperplane.
A *ball* centered at c with radius r is a set of the form

$$\{x \mid \|x - c\| \leq r\}.$$

Because norms are convex, balls are convex sets.
An important class of sets in MPC is given by *polytopes*. These are intersections of finitely many halfspaces:

$$\{x \mid a_i^\top x \leq b_i,\ i = 1, \ldots, r\} = \{x \mid Ax \leq b\}.$$

State and input constraints in linear MPC are usually expressed exactly in this form.
It is useful to distinguish between unions and intersections. The intersection of convex sets is always convex, because any line segment that remains in each set separately also remains in their common intersection. In contrast, the union of convex sets is not necessarily convex. Two disjoint intervals on the real line already provide a counterexample. This simple observation is important in practice: adding constraints tends to preserve convexity, while combining alternative feasible regions often destroys it.

A key geometric property of convex sets is the existence of supporting hyperplanes. Intuitively, a supporting hyperplane touches the boundary of a convex set without cutting through it.
Suppose a set is described as

$$X = \{x \mid g(x) \leq 0\},$$

and let z be a boundary point such that $g(z) = 0$ and g is differentiable at z. Then a local supporting hyperplane is given by

$$\nabla g(z)^\top (x - z) \leq 0.$$

Equivalently,

$$\nabla g(z)^\top x \leq \nabla g(z)^\top z.$$

This is the first-order linear approximation of the boundary at the point z. The result explains why affine constraints are so natural in optimization: they describe exactly the geometry of tangent or supporting hyperplanes.
The picture becomes more delicate when the boundary has a kink and the gradient is not defined. In that case, one has to replace the gradient by a more general notion such as a subgradient. For the purposes of this handout, the differentiable case is the most relevant one.

Several classes of convex functions occur repeatedly in control.

Affine functions. Any function of the form

$$f(x) = a^\top x + b$$

is convex. In fact, affine functions are both convex and concave.

Norms. Functions of the form

$$f(x) = \|x - c\|$$

are convex. This explains why norm-based penalties are common in estimation, robust control, and optimization.

Quadratic functions. A quadratic function

$$f(x) = x^\top A x + b^\top x + c$$

is convex if and only if $A \succeq 0$, that is, if A is positive semidefinite. This is the basic reason why quadratic costs in LQR and MPC lead to convex optimization problems when the constraints are also convex.

A.2 Derivative-based characterizations of convexity

For differentiable and twice differentiable functions, convexity can be tested through derivatives. If f is differentiable on a convex domain, then f is convex if and only if

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) \qquad \forall x, y.$$

This means that the tangent hyperplane at any point underestimates the function globally. Equation (A.2) is one of the most useful characterizations of convexity, both conceptually and algorithmically.
If f is twice differentiable on a convex domain, then f is convex if and only if

$$\nabla^2 f(x) \succeq 0 \qquad \forall x.$$

Thus, convexity can be checked by testing whether the Hessian is positive semidefinite everywhere. In one dimension, this reduces to the familiar condition $f''(x) \geq 0$. In several dimensions, the Hessian plays the same role, but now curvature must be nonnegative in every direction.

A.3 Convex optimization problems

We now return to the constrained problem in (3). It is called a *convex optimization problem* if

- X is a convex set,
- f is convex,
- each inequality constraint function g_i is convex,
- each equality constraint is affine.

Under these assumptions, the feasible set

$$X \cap \{x \mid g(x) \leq 0\} \cap \{x \mid h(x) = 0\}$$

is convex. The reason the equality constraints must be affine is that a general nonlinear equality constraint would typically define a nonconvex set.
Convex optimization problems are much simpler than general nonlinear programs for a fundamental reason: there are no spurious local minima. This is the main structural property that makes them attractive in real-time control.

A point $x^\star$ is a local minimum if there exists $R > 0$ such that

$$f(z) \geq f(x^\star)$$

for all feasible z satisfying $\|z - x^\star\| \le R$.
For convex problems, local optimality automatically implies global optimality. The proof is short and very instructive. Assume that $x^\star$ is locally optimal but not globally optimal. Then there exists a feasible point y such that

$$f(y) < f(x^\star).$$

Because the feasible set is convex, every point on the segment

$$z_t = ty + (1-t)x^\star, \qquad t \in [0,1]$$

is feasible. Choosing $t > 0$ small enough, we can place z_t inside the ball of radius R around $x^\star$. By convexity of f,

$$f(z_t) \le tf(y) + (1-t)f(x^\star) < f(x^\star),$$

which contradicts local optimality. Hence every local minimum is global.
This single fact explains much of the power of convex optimization: once a candidate point is known to be locally optimal, the search is over.

A.4 First-order optimality conditions

Suppose the problem is differentiable. Then a feasible point $x^\star$ is locally optimal if and only if

$$\nabla f(x^\star)^\top (y - x^\star) \ge 0 \qquad \text{for all feasible } y. \tag{4}$$

This condition says that, at an optimum, the gradient cannot point toward any feasible descent direction.
In the unconstrained case, every direction is feasible, so (4) reduces to

$$\nabla f(x^\star) = 0.$$

Thus, unconstrained optimization leads to the familiar linear algebra test of setting the gradient equal to zero.
The constrained case is more interesting. At the optimum, the negative gradient may point outside the feasible set, so it cannot be canceled by free motion of the decision variable. Instead, it must be balanced by the geometry of the active constraints. This leads to the Karush–Kuhn–Tucker conditions.

A.5 KKT conditions

Consider the constrained problem

$$\begin{aligned} \min_{x \in X} \quad & f(x) \\ \text{s.t.} \quad & g(x) \le 0, \\ & h(x) = 0. \end{aligned}$$

Under suitable regularity assumptions, a local optimum $x^\star$ satisfies the KKT conditions: there exist multiplier vectors $\mu \in \mathbb{R}^m$ and $\lambda \in \mathbb{R}^p$ such that

$$\nabla f(x^\star) + \sum_{i=1}^{m} \mu_i \nabla g_i(x^\star) + \nabla h(x^\star)^\top \lambda = 0, \tag{5}$$

$$g_i(x^\star) \le 0, \qquad i = 1, \dots, m, \tag{6}$$

$$h(x^\star) = 0, \tag{7}$$

$$\mu_i \ge 0, \qquad i = 1, \dots, m, \tag{8}$$

$$\mu_i g_i(x^\star) = 0, \qquad i = 1, \dots, m. \tag{9}$$

Each condition has a clear meaning. Equation (5) is the stationarity condition: the gradient of the cost is balanced by a linear combination of the gradients of the active constraints. Conditions (6) and (7) express primal feasibility. Condition (8) says that inequality multipliers must be nonnegative. Finally, (9) is the complementary slackness condition: if a constraint is inactive, then its multiplier is zero; if its multiplier is positive, then the constraint must be active.
For convex optimization problems, these conditions are especially powerful. Under standard constraint qualifications, they are not only necessary but also sufficient. In other words, solving the KKT system is enough to recover the global optimizer.

A.6 Lagrangian viewpoint

The KKT conditions can be organized through the Lagrangian

$$\mathcal{L}(x, \mu, \lambda) := f(x) + \mu^\top g(x) + \lambda^\top h(x).$$

From this viewpoint, constrained optimization can be interpreted as the search for a saddle point of the Lagrangian: one minimizes with respect to the primal variable x and maximizes with respect to the dual variables associated with the inequalities and equalities.
This viewpoint is central in many numerical methods. It is also conceptually useful in control, because dual variables often admit an interpretation as shadow prices, sensitivities, or integral correction signals. This becomes particularly visible in primal–dual algorithms and dual ascent methods.

There are two complementary ways to think about convex optimization.
The first is the *algebraic view*. Since the KKT conditions characterize the optimum, one may try to solve a structured system of equations involving the primal variable and the multipliers. In quadratic programming, this often leads to linear algebraic systems with a very specific sparsity pattern.
The second is the *geometric or iterative view*. Since the gradient provides a descent direction and convexity rules out bad local minima, one can use iterative algorithms such as gradient descent, projected gradient descent, dual ascent, or interior-point methods. These algorithms repeatedly apply simple updates until convergence.

This is exactly where the earlier observation becomes relevant again: computers are good at linear algebra and iteration. Convex optimization exploits both. The geometry guarantees that the iterates move toward the correct solution, while the algebraic structure makes each update computationally efficient. This is the reason convex programs can often be solved fast enough for online control.

A.7 Connection with MPC and control

The relevance of convex optimization for control can now be summarized clearly.
In unconstrained LQR, the optimization problem is quadratic and unconstrained, so the optimum can be computed analytically through the Riccati recursion. In constrained MPC with linear dynamics, quadratic cost, and polyhedral state and input constraints, the online problem is a convex quadratic program. Because the problem is convex, the optimizer is globally meaningful, the value function is well behaved, and the explicit MPC law becomes piecewise affine when it is computed offline.
More generally, many feedback-optimization schemes rely on repeatedly solving simple convex subproblems. Projection steps, local quadratic approximations, and primal–dual iterations are all practical only because the underlying subproblems preserve convexity.